

**An Energy-Centric Comparative Evaluation of Software-Based vs.
Hardware-Accelerated Cryptography on ESP32 in a Realistic IoT
Duty Cycle**

Submitted By:

Samoda Wijesooriya

IM/2021/059

Supervised By:

Dr. Keerthi Wijayasiriwardhane

A dissertation submitted in partial fulfilment of the requirements for the Degree of
Bachelor of Science in Information Technology

Department of Industrial Management

Faculty of Science

University of Kelaniya

Sri Lanka

Sep 2026

Declaration

I hereby state that this research report and findings presented in it are my own and it has not been submitted before nor is it currently being submitted for any other academic programme. Where material has been used from other sources due recognition has been given by mentioning the source.

Signature of the student

Signature: _____

Date: 22/09/2026

Signature of Supervisor

Supervisor's Signature: _____

Date: _____

Abstract

The Internet of Things increasingly relies on battery-powered devices that must balance data security against strict energy limits, yet no reviewed study compares software-based lightweight cryptography with hardware-accelerated legacy cryptography on the same platform under duty-cycle conditions. This dissertation evaluates ASCON-128, the NIST lightweight standard implemented in software, against AES-128-GCM using the dedicated AES hardware engine of the ESP32, in a Sleep $\rightarrow$ Wake $\rightarrow$ Sense $\rightarrow$ Encrypt $\rightarrow$ Transmit $\rightarrow$ Sleep duty cycle in which the encryption phase is measured on the device and the sleep phase is modelled from datasheet values.

Two measurement campaigns were carried out on an ESP32-WROOM-32E instrumented with an INA219 current monitor. A per-trial campaign (100 payload sizes, 16 to 1,600 bytes; 561 and 510 complete iterations) was found to be limited by instrumentation: the sensor cannot resolve a roughly 2 ms operation, and the timed region included task creation and sensor reads. A sustained-load campaign was therefore designed, in which each cipher call is repeated back-to-back while the sensor averages current, and the two algorithms are interleaved within one session. Its results are used for all energy conclusions.

Setting up the AES-GCM context cost 30.9 μ s and 1.9 μ J per call. When that setup is paid on every message, as on a node that loses RAM in deep sleep, ASCON-128 is more energy-efficient below a break-even payload of approximately 100 bytes (95% bootstrap interval 96–107 bytes), and AES-128-GCM is more efficient above it; at 1,600 bytes AES-128-GCM used 28% less energy per call (104.7 μ J versus 145.2 μ J). If the AES context is kept between messages, AES-128-GCM was at least as efficient at every tested size. AES-128-GCM also used about 24% more memory. Against a modelled 60-second sleep interval, the choice of cipher changed total duty-cycle energy by less than 4%.

These findings reconcile a conflict in the literature and give designers a payload-size and setup-policy criterion for choosing between software and hardware cryptography.

Keywords: Internet of Things (IoT); Lightweight Cryptography; ASCON-128; AES-128-GCM; ESP32; Hardware Acceleration; Energy Efficiency; Duty Cycle; Break-Even Payload Size; Authenticated Encryption (AEAD)

Acknowledgements

I sincerely thank everyone who supported me throughout this research.

My deepest gratitude goes to my supervisor, Dr. Keerthi Wijayasiriwardhane, for his continuous guidance, encouragement, and technical supervision, which were vital to the successful completion of this study.

I also extend my appreciation to the research presentation panel, especially Dr. Dilani Wickramaarachchi, for their valuable feedback that helped refine this work.

I am thankful to Mr. Ishara Meegahawela, my senior at Binarywise (Pvt) Ltd, for his support and flexibility for their encouragement during this period.

Finally, I am deeply grateful to my friends, parents for their encouragement, understanding, and constant motivation, which inspired me to complete this journey successfully.

Contents

Declaration	ii
Abstract	iii
Acknowledgements	iv
List of figures	ix
List of Tables	ix
List of Equations	x
Abbreviations	xi
Chapter 01: Introduction	1
1.1 Overview of the Internet of Things (IoT)	1
1.2 Cryptographic Requirements in Constrained Environment	1
1.3 The Energy-Security Trade-off Software vs Hardware Acceleration	2
1.4 The Importance of Realistic IoT Duty Cycles	2
1.5 Problem in Existing IoT Implementations	3
1.6 Need for Comparative Evaluation	3
1.7 Research Questions	4
1.8 Objectives	4
1.9 Organization of Dissertation	5
Chapter 02: Literature Review	6
2.1 Introduction to Cryptography and IoT Security	6
2.1.1 The Evolution of Data Security and Encryption	6
2.1.2 IoT Threat Landscape and Device Constraints	7
2.2 Cryptographic Foundations for IoT	7
2.2.1 Symmetric vs Asymmetric Encryption	7
2.2.2 Role of AEAD in IoT Security	8
2.2.3 Common IoT Encryption Methods	8
2.2.4 AES and AES-GCM	9
2.2.5 Lightweight Cryptography: The ASCON Standard	10
2.3 Software-Based vs Hardware-Accelerated Cryptography	10
2.3.1 Principles and Trade-offs of Software Cryptography	10
2.3.2 Principles and Trade-offs of Hardware Cryptographic Engines	11
2.3.3 The ESP32 Hardware Accelerator	11

2.3.4 Comparative Literature: Software ASCON vs Hardware AES	12
2.4 Evaluation Methodologies for Energy-Centric Benchmarking.....	13
2.4.1 Metrics and Benchmarking Practices	13
2.4.2 Measurement Techniques and Instrumentation	14
2.5 Realistic IoT Duty Cycles and Energy Implications	15
2.5.1 Duty Cycle Structure (Sleep → Wake → Sense → Transmit → Sleep)	15
2.5.2 Sleep Implementation Mechanisms.....	16
2.6 Synthesis of Comparative Evidence.....	17
2.7 Research Gap and Justification	18
Chapter 03: System Design and Methodology	19
3.1 Research Process	19
3.1.1 Research Methodology	19
3.2 Selection of Cryptographic Algorithms	20
3.2.1 Rationale for AES-128-GCM	20
3.2.2 Rationale for ASCON-128.....	20
3.2.3 Common Evaluation Basis	21
3.3 Experimental Design.....	21
3.3.1 Overview of the Measurement Program.....	22
3.3.2 Payload Generation Strategy	24
3.3.3 Nonce Generation	24
3.3.4 Duty-Cycle/ Sleep Design	24
3.4 Test-bed Implementation.....	25
3.4.1 Hardware Components	26
3.4.2 Software Environment and Libraries	28
3.4.3 Schematic Diagram.....	29
3.4.4 Wiring Diagram and Physical Setup.....	30
3.5 Measurement Methodology.....	31
3.5.1 Execution Time Measurement	31
3.5.2 Throughput Calculation	31
3.5.3 Power Consumption Measurement.....	32
3.5.4 Memory Footprint Measurement.....	32
3.6 Data Collection and Extraction	33
3.7 Expected Data Analysis and Statistical Evaluation.....	33

3.7.1 Statistical Methods and Tools	33
3.7.2 Significance/Comparison Method	34
Chapter 04: Data Analysis and Results	35
4.1 Introduction	35
4.2 Data Collection Outcome and Cleaning Summary	36
4.2.1 Raw Log Volumes and Structural Completeness.....	36
4.2.2 INA219 Zero-Offset Calibration	37
4.2.3 Outlier Trimming	38
4.2.4 Sleep-Phase Energy Constant	38
4.3 Execution Time and Throughput Results	39
4.3.1 Execution Time Scaling.....	39
4.3.2 Throughput	39
4.4 Power Consumption Results	40
4.5 Per-Byte Energy Efficiency and Break-Even Analysis	41
4.5.1 Per-Byte Energy Curves	41
4.5.2 Break-Even / Crossover Payload Size	42
4.5.3 Regime Segmentation.....	43
4.6 Memory Footprint Results	44
4.7 Statistical Significance Testing	45
4.7.1 Per-Size Hypothesis Testing	45
4.7.2 Regression Summary	45
4.8 Summary of Chapter 4	45
Chapter 05: Validation of the Measurements.....	46
5.1 Introduction	46
5.2 Why Validation Was Needed: Audit of Campaign (A)	46
5.2.1 Method.....	46
5.2.2 Findings	46
5.3 Validation Design (Campaign B)	48
5.3.1 Principle.....	48
5.3.2 Procedure	49
5.3.3 Variants	49
5.3.4 Runs	50
5.3.5 Statistics.....	50

5.4 Validation Results.....	50
5.4.1 Data quality.....	50
5.4.2 Time per operation and throughput	50
5.4.3 Setup cost of the AES-GCM context.....	51
5.4.4 Extra power.....	52
5.4.5 Energy per operation and per byte	52
5.4.6 Break-even payload size	53
5.4.7 Effect of reusing the AES context	55
5.4.8 Encryption within a modelled duty cycle	55
5.5 Re-examination of the Campaign A Results	56
5.5.1 Side-by-side comparison	56
5.5.2 The origin of the 347-byte break-even	58
5.5.3 Regression versus measured break-even	58
5.5.4 The meaning of the regression intercept.....	59
5.5.5 Outlier trimming.....	59
5.5.6 Statistical testing.....	60
5.6 Reliability and Validity of the Validation	61
5.8 Summary of Chapter 5	62
Chapter 06: Discussion	63
6.1 Introduction	63
6.2 Interpretation of Findings.....	63
6.2.1 Why a break-even point exists at all.....	63
6.2.2 Objective 2 - Energy Cost Per Transmitted Byte Across the Full Duty Cycle	65
6.2.3 Power versus time.....	65
6.2.4 Why the first analysis gave 347 bytes	66
6.3 Comparison with Existing Literature	66
6.4 Limitations	67
6.5 Summary of Chapter 6	68
Chapter 07: Conclusions and Future Work	69
7.1 Introduction	69
7.2 Achievement of Research Objectives.....	69
7.3 Answer to the Research Question	70

7.4 Contribution to Knowledge	70
7.5 Recommendations for Future Work	70
7.6 Closing Remarks	71
8. References	72

List of figures

Figure 1 - Main Algorithm	23
Figure 2 - Functional Block Diagram of ESP32	26
Figure 3 - Simplified Schematic of INA219	27
Figure 4 - Test Bed Side View	28
Figure 5 - Schematic Diagram	29
Figure 6 - Test Bed	30
Figure 7- execution time scaling	39
Figure 8-Regression scaling overlay	40
Figure 9- Energy per byte comparison	42
Figure 10-Total duty cycle energy per byte	42
Figure 11- Memory footprint comparison	44
Figure 12 - Sustained load measurement of one configuration	48
Figure 13 - Sensor timing scale	48
Figure 14 - Time per operation	51
Figure 15 - AES setup cost	51
Figure 16 - Power consumption by variant	52
Figure 17 - Energy per operation with linear fits (Run 1)	53
Figure 18 - Energy per byte against payload size	54
Figure 19 - Fine sweep, run 2	55
Figure 20 - Encryption share of the modelled cycle at each payload cycle	56
Figure 21 - Campaign A's stepped power/energy vs Campaign B's flat	58
Figure 22 - Break even under three trimming choices (none, $k=3$, $k=1.5$)	60

List of Tables

Table 1 - Common IoT Encryption Methods	9
Table 2 - Summary of Evaluation Metrics and Methodologies	21
Table 3-Row counts per cleaning stage	37
Table 4-Time and energy regression results	40
Table 5-Comparison by size	43
Table 6-Regime segmentation	44
Table 7- Audit of the Campaign A measurement method	47
Table 8-Cipher variants in Campaign B	49
Table 9- Time per operation	51

Table 10- Energy per operation and per byte	52
Table 11- Linear fits of energy per call against payload size	52
Table 12- Fine sweep around the break-even	53
Table 13 - Effect of the cipher on modelled cycle energy	56
Table 14 - Time per operation in Campaign A and Campaign B	56
Table 15 - Extra power and energy per operation in Campaign A and Campaign B ...	57
Table 16 - Sensitivity of campaign A to outlier trimming	59
Table 17 - Campaign A per-size tests	60
Table 18- Resource cost before encryption	64

List of Equations

Equation 1 - Throughput Equation	32
Equation 2 - Total Power Equation	32
Equation 3- Sleep-phase energy constant	38
Equation 4 - Energy per call equation	49

Abbreviations

ADC - Analog-to-Digital Converter

AEAD - Authenticated Encryption with Associated Data

AES - Advanced Encryption Standard

AES-GCM - Advanced Encryption Standard, Galois/Counter Mode

API - Application Programming Interface

ASIC - Application-Specific Integrated Circuit

CI - Confidence Interval

CPU - Central Processing Unit

DMA - Direct Memory Access

ECC - Elliptic Curve Cryptography

ESP-IDF - Espressif IoT Development Framework

FPGA - Field-Programmable Gate Array

GCM - Galois/Counter Mode

GDMA - General Direct Memory Access

GHASH - Galois Hash (authentication function used in GCM)

GPIO - General-Purpose Input/Output

I2C - Inter-Integrated Circuit

INA219 - Current/Power Monitor Sensor (Texas Instruments)

IoT- Internet of Things

IQR - Interquartile Range

LPWAN - Low-Power Wide-Area Network

LWC - Lightweight Cryptography

MQTT - Message Queuing Telemetry Transport

NIST - National Institute of Standards and Technology

PRNG - Pseudo-Random Number Generator

RAM - Random Access Memory

RF - Radio Frequency

RSA - Rivest–Shamir–Adleman

RTC - Real-Time Clock

SoC - System-on-Chip

SRAM - Static Random-Access Memory

ULP - Ultra-Low-Power (coprocessor)

Chapter 01: Introduction

1.1 Overview of the Internet of Things (IoT)

The Internet of Things (IoT) represents a transformative paradigm in modern technology, characterized by a vast network of physical objects, or "things," embedded with sensors, software, and other technologies that connect and exchange data with other devices and systems over the Internet [F. Thabit, 2023]. The exponential growth of IoT has been remarkable; the number of connected IoT devices globally reached 21.1 billion in 2025 and is projected to surge to nearly 39 billion by 2030 [Sinha, 2025]. These smaller embedded devices operate within constrained environments, often in remote or inaccessible locations such as agricultural fields, industrial machinery, or environmental monitoring stations. Because of these devices are frequently deployed in inaccessible or hazard locations, replacing or recharging their batteries is highly impractical. This massive interconnected device necessitates robust and secure communication channels to protect the sensitive data being transmitted across heterogeneous networks. Consequently, optimizing the consumption of every available resource particularly energy is a primary design imperative in IoT systems.

1.2 Cryptographic Requirements in Constrained Environment

As IoT devices increasingly handle sensitive data and control critical infrastructure, securing their communication channels has become more important than anything else. Unsecured IoT networks are highly vulnerable to eavesdropping, data tampering, and device spoofing [Sadhu et al., 2022]. To ensure confidentiality, integrity, and authenticity, cryptographic primitives such as Advanced Encryption Standard (AES), Secure Hash Algorithm (SHA) families, and emerging Lightweight Cryptography (LWC) standards like ASCON are employed [Turan et al., 2025].

However, the mathematical complexity inherits in these encryptions and hashing algorithms demand significant computational overhead. Traditional cryptographic standards, while highly secure, were designed for resource-rich environments and can rapidly consume the limited energy reserves of a constrained IoT devices if not carefully implemented [Kumar & Sethi Ericsson, 2019]. These necessitates a careful

evaluation of which cryptographic approaches yield the optimal balance between security and energy efficiency.

1.3 The Energy-Security Trade-off Software vs Hardware Acceleration

A critical factor in the energy consumption of IoT cryptography is the implementation method: software-based execution vs hardware-accelerated execution. Microcontrollers like the ESP32 features a dual-core processor capable of running cryptographic algorithms purely in software. Alternatively, modern System-on-Chip (SoC) designs often include dedicated hardware cryptographic accelerators, specialized silicon blocks designed to execute specific algorithms (like AES or SHA) much faster than the main CPU [Jin & Becker, 2022].

While hardware acceleration theoretically reduces the active execution time of cryptographic operations, it introduces additional wake-up latencies and may draw higher peak currents. Conversely, highly optimized software implementations might run longer but could maintain a lower, more stable power draw. Evaluating whether the time-savings of hardware acceleration actually translate to net energy savings requires a granular, energy-centric analysis rather than a simple speed benchmark [Kietzmann et al., 2021]. For instance, research on ESP32 hardware acceleration has shown significant throughput improvements (e.g., +257.8% for AES encryption), but the total energy impact across a full duty cycle remains a critical area of study [Jin & Becker, 2022].

1.4 The Importance of Realistic IoT Duty Cycles

Historically, many cryptographic evaluations on IoT devices measure only the active execution time or the average current draw during continuous operations. However, this does not reflect a realistic IoT duty cycle. In practical scenarios IoT devices spend the majority of their time in deep sleep mode to conserve battery, waking up only briefly to sense data, perform cryptographic operations, and transmit payloads [Khriji et al., 2022].

In a realistic duty cycle, the energy overhead of transitioning from sleep to activate states, the idle leakage current during the cryptographic execution, and the baseline sleep current of the microcontroller become dominant factors. Therefore, an accurate

evaluation of cryptographic energy efficiency must be conducted within the context of a full, realistic IoT duty cycle, measuring the total energy consumed from the moment the device wakes up until it return to deep sleep [Kozłowski & Sosnowski, 2019]. For example, a typical sensor node waking up every 5 minutes may spend only 3 seconds active (drawing $\sim 100\text{mA}$) and 297 seconds in deep sleep (drawing $\sim 10\mu\text{A}$), making the cryptographic overhead during the active phase highly influential on the overall battery life [Community, 2026].

1.5 Problem in Existing IoT Implementations

Although modern microcontrollers such as the ESP32 include dedicated hardware encryption accelerators, much IoT security research continues to emphasize lightweight software-based cryptography such as ASCON. While lightweight ciphers are designed for portability and reduced computational overhead, they remain CPU-bound and can impose significant energy and performance costs within realistic IoT duty cycles. This creates a paradox: developers and researchers prioritize lightweight cryptography for constrained devices yet may overlook built-in hardware engines that can offload encryption tasks, reduce latency, and improve energy efficiency. The lack of systematic studies comparing hardware-accelerated AES-128-GCM with software-based lightweight cryptography (ASCON-128) across realistic IoT duty cycles leaves a critical gap in understanding the true trade-offs among per-byte energy consumption, execution time, throughput, and memory usage on the ESP32 platform.

1.6 Need for Comparative Evaluation

Despite the availability of hardware encryption accelerators in microcontrollers such as the ESP32, systematic comparisons with modern software-based lightweight cryptography across full IoT duty cycles remain limited. While traditional standards like AES-128 GCM are widely deployed and hardware-optimized for speed, they can still be resource-intensive in terms of power and memory depending on the implementation [Radhakrishnan et al., 2024]. In contrast, ASCON-128 has been selected as the NIST primary standard for lightweight cryptography specifically because its 320-bit permutation-based sponge architecture is designed to offer robust Authenticated Encryption with Associated Data (AEAD) with minimal overhead in constrained environments [Kandi et al., 2023]. This research addresses this gap by

evaluating whether software-based ASCON can outperform hardware-accelerated AES in low-power, battery-operated IoT nodes [Radhakrishnan et al., 2024].

This study builds upon the methodology established by Amirkhanova et al. (2026a), who demonstrated a practical end-to-end encrypted pipeline using AES-GCM on the ESP32 but identified critical limitations in their own work, including a narrow benchmark scope and a failure to quantify energy and CPU utilization. By expanding this scope, this research implements a realistic IoT duty cycle consisting of **Sleep** → **Wake** → **Sense** → **Encrypt** → **Transmit** → **Sleep** which accurately reflects how real-world devices manage their limited battery life rather than operating continuously [Ciuffoletti, 2022], [Hatch, 2025]. Furthermore, this study directly addresses the future work suggested by prior literature by providing granular, per-byte energy and CPU analysis [Amirkhanova et al., 2026].

1.7 Research Questions

This study seeks to address the following central question: How does hardware-accelerated AES-128-GCM on the ESP32 compare with software-based ASCON-128 in terms of power consumption, memory usage, throughput, execution time, and per-byte energy efficiency when evaluated across a full IoT duty cycle (**Sleep** → **Wake** → **Sense** → **Encrypt** → **Transmit** → **Sleep**)?

By answering this question, the research aims to determine whether leveraging the ESP32’s built-in cryptographic accelerators provide a more energy-efficient and practical security solution for IoT devices than relying solely on lightweight software cryptography.

1.8 Objectives

- To compare the power consumption (total duty-cycle energy) and per-byte energy efficiency of ASCON-128 implemented in software with AES-128-GCM using hardware acceleration on the ESP32 platform, including the initialization overhead of the hardware engine.
- To measure the energy cost per transmitted byte across a realistic IoT duty cycle (**Sleep** → **Wake** → **Sense** → **Encrypt** → **Transmit** → **Sleep** for both cryptographic approaches.

- To determine the conditions (e.g., payload size, duty-cycle frequency, and available energy budget) under which software-based ASCON-128 remains preferable despite the availability of hardware-accelerated AES-128-GCM, including identifying any break-even (crossover) payload size where one approach becomes more energy efficient than the other.

1.9 Organization of Dissertation

This dissertation is systematically organized into distinct chapters, each serving a specific purpose in presenting the research work comprehensively:

Chapter 1: Introduction – Introduces the fundamental concepts of IoT and highlights the critical role of secure communication in constrained environments. It establishes the research problem by framing the energy–security trade-off in cryptographic implementations and sets the foundation for the comparative evaluation presented in subsequent chapters.

Chapter 2: Literature Review – Provide a survey the existing body of the research on IoT security, cryptographic standards, and evaluation methodologies. It synthesizes prior findings to highlight the unresolved trade-offs between lightweight software cryptography and hardware-accelerated AES within realistic IoT duty cycle.

Chapter 3: Methodology – Outlines the experimental methodology, detailing the IoT duty-cycle testbed, encryption pipeline, and measurement setup for comparing hardware-accelerated AES-128-GCM with software-based ASCON-128.

Chapter 4: Data Analysis and Results - Reports the per-trial benchmark (Campaign A): cleaning, time, power, energy, break-even, memory and statistical tests.

Chapter 5: Validation of the Measurements - Audits the measurement method of Campaign A, reports a second, sustained-load campaign (Campaign B), and reassesses the Campaign A results, including the responses to the review comments.

Chapter 6: Discussion - Interprets the corrected findings, compares them with the literature and states the limitations.

Chapter 7: Conclusions and Future Work - Answers the research question, states the contribution and lists recommendations.

Chapter 02: Literature Review

2.1 Introduction to Cryptography and IoT Security

Cryptography has evolved from ancient historical methods to standardized digital algorithms that provide the essential framework for securing modern global communications. Symmetric encryption became the cornerstone of modern data protection following the establishment of the Advanced Encryption Standard (AES) by NIST in 2001. These cryptographic primitives are designed to uphold the CIA triad confidentiality, integrity, and availability by serving as a primary defense against eavesdropping and man-in-the-middle attacks across wireless and cloud infrastructures. The ongoing transition toward post-quantum algorithms reflects the critical need for cryptography to adapt to emerging threats while safeguarding the integrity of digital societies.

A major constraint of these traditional standards is that they were primarily optimized for high-performance desktop and server environments, where computational resources and power are abundant. Applying these heavy digital standards directly to the limited architectures of the Internet of Things creates a significant performance gap that requires specialized solutions.

2.1.1 The Evolution of Data Security and Encryption

As security requirements extended beyond desktop and server environments into resource-constrained IoT deployments, the field began shifting away from a "one-size-fits-all" cryptographic model. Rather than treating AES and RSA as universally deployable standards, researchers increasingly recognized that device class not just threat model should determine algorithm choice. This shift gave rise to a distinct research effort around lightweight cryptography (LWC), purpose-built for constrained hardware rather than adapted from server-grade primitives after the fact. The specific mechanics and measured performance of this shift what LWC standards like ASCON actually achieve, and how they compare to legacy alternatives are examined in detail in 2.2, since evaluating those claims requires first establishing the constraints they are designed to solve.

2.1.2 IoT Threat Landscape and Device Constraints

The rapid expansion of the Internet of Things has created a massive attack surface composed of billions of heterogeneous devices that operate under severe resource constraints, significantly increasing their vulnerability to exploitation. IoT endpoints typically utilize low-power microcontrollers clocked between 8 MHz and 32 MHz with on-chip RAM restricted to the 4 KB to 64 KB range, and even capable platforms like the dual-core 240 MHz ESP32 are frequently deployed under minimalist conditions [Faisal Abdullah Althobaiti, 2025]. These devices must often sustain operations for months or years on small coin-cell (~100 mAh) or Li-ion batteries without maintenance, yet executing traditional RSA-2048 operations can consume several seconds of processor time and rapidly deplete energy reserves. Furthermore, because these nodes are often deployed in unstaffed environments and linked via wireless protocols like LoRa or WiFi, they are inherently exposed to eavesdropping and physical tampering, while traditional transport-layer security often fails to account for the massive energy overhead of complex handshakes or the risk posed by untrusted MQTT brokers [Bideh et al., 2020].

A critical limitation in the current literature is the frequent reliance on software simulations or analytical models that omit physical hardware factors such as start-up energy costs and realistic hardware duty cycles, rather than measurements taken on real, deployed hardware [Nkemeni et al., 2023; Dezfouli et al., 2018]. This gap between simulated and real-world energy behavior is not incidental.

Because resources are limited, security features are often turned off to make devices last longer. This raises the question of which cryptographic method works best under such limits.

2.2 Cryptographic Foundations for IoT

2.2.1 Symmetric vs Asymmetric Encryption

Symmetric and asymmetric encryption offer distinct trade-offs between performance and key management scalability for IoT environments. Symmetric ciphers are generally preferred for low-power IoT implementations because they utilize a single shared key for rapid encryption and decryption with minimal computational overhead [Agugliaro et al., 2024]. In contrast, asymmetric methods like RSA or ECC improve key

management but require significant processing time and memory, often exceeding the budgets of the most constrained devices [Ibrahim et al., 2025]. Current literature indicates that while 256-bit ECC offers security comparable to RSA-3072, the computational cost of asymmetric operations remains thousands of times higher than symmetric block ciphers on 8-bit or 16-bit microcontrollers [Faisal Abdullah Althobaiti, 2025].

A significant drawback cited by researchers is the "k Kings puzzle" of secure key distribution at scale for symmetric schemes, while pure asymmetric solutions are acknowledged as prohibitively energy-consuming for autonomous sensor nodes [Ibrahim et al., 2025]. This imbalance the need of a hybrid approach where asymmetric primitives handle trust establishment and symmetric primitives manage bulk data protection.

2.2.2 Role of AEAD in IoT Security

Authenticated Encryption with Associated Data (AEAD) provides a unified mechanism for ensuring both data secrecy and message integrity within a single cryptographic operation AEAD is critical for IoT because common application protocols like CoAP and MQTT do not include native security features, leaving payloads vulnerable to interception or tampering [Bideh et al., 2020]. By processing associated data; such as network headers, alongside the payload in a single pass, AEAD schemes like AES-GCM or ASCON-128 fulfill confidentiality and authenticity requirements more efficiently than traditional two-pass constructions that require separate HMAC functions [Padmanabha1 et al., 2025]. On ARM Cortex-M0+ platforms, these integrated suites have been shown to reduce cryptographic energy burdens from roughly 35% to under 8% of the total energy budget [Albert, 2026]. While authors claim AEAD improves efficiency, they note that the resulting ciphertext expansion and tag overhead can still impact performance in narrow-band LPWAN technologies where payloads are extremely small [Faisal Abdullah Althobaiti, 2025]. The transition from basic block ciphers to AEAD suites is central to modern IoT security standards.

2.2.3 Common IoT Encryption Methods

Algorithm	Type	Security Strength	Primary focus/ Trade-off

AES-128	Symmetric (Block)	128-bit	Legacy standard; robust but resource-intensive in software
RSA-2048	Asymmetric	High	Standardized PKI; prohibitive energy drains for IoT
ECC (Curve25519)	Asymmetric	High	Efficient key exchange; better than RSA but still slower than symmetric
SPECK-64/128	Symmetric (Block)	128-bit	Software-oriented; ultra-low resource requirements
ASCON-128	Symmetric (AEAD)	128-bit	Purpose-built LWC standard; minimal memory and power

Table 1 - Common IoT Encryption Methods

This table reveals a clear cluster around AEAD-capable symmetric algorithms, which prioritize reducing the "security-performance gap" for real-time sensing. While asymmetric methods like ECC solve identity and key distribution problems, they represent a distinct trade-off of high memory usage and latency that limits their use to infrequent handshake operations [Agugliaro et al., 2024]. This pattern underscores a research shift toward purpose-built lightweight symmetric standards like ASCON over legacy desktop-oriented algorithms.

2.2.4 AES and AES-GCM

The Advanced Encryption Standard remains the most widely trusted symmetric primitive, though its iterative design claims were originally optimized for high-performance processors. Standardized by NIST in 2001, AES utilizes a 128-bit state matrix and a 10-round schedule of complex algebraic transformations to provide robust defense against differential and linear cryptanalysis [Farooq et al., 2020]. When combined with the Galois Counter Mode (GCM), it offers high-speed authenticated encryption through binary field multiplication, though it requires significant RAM and CPU cycles that can rapidly deplete coin-cell batteries [Kim et al., 2020]. Design-level claims emphasize the algorithm's agility and simplicity for hardware-independent security [Farooq et al., 2020].

Despite its design claims, researchers acknowledge that standard software-emulated AES on low-power microcontrollers lacking native hardware vector units introduces

severe cycle execution overhead and high latency [Ridho Rabbani, 2025]. This makes AES a heavyweight-but-accelerable legacy option that often relies on dedicated silicon to bridge its performance gap.

2.2.5 Lightweight Cryptography: The ASCON Standard

ASCON was purpose-built to provide the highest levels of security within the minimal resource envelopes typical of low-power sensor nodes. Selected as the NIST lightweight standard in 2023, the ASCON family utilizes a 320-bit permutation-based sponge architecture that performs AEAD and hashing in a single pass [Turan et al., 2025]. ASCON claim that it balances throughput and memory use by employing bit-sliced 5-bit S-boxes, which reduces hardware gate counts compared to the 8-bit S-boxes used in AES [Ridho Rabbani, 2025]. It is intended for devices with less than 32 KB of RAM, aiming for a code footprint below 10 KB [Faisal Abdullah Althobaiti, 2025].

While ASCON focus on optimizing efficiency through lightweight permutation rounds, practical measurements show that its sponge duplexing introduces linear runtime scaling [Ridho Rabbani, 2025]. As payload increase, this iterative structure can lead to slower performance in extended bulk stream operations when hardware acceleration is not available.

The extend to which of these algorithm-level characteristics are realized is strongly influenced by the mode of implementation, whether executed in software or supported through dedicated hardware.

2.3 Software-Based vs Hardware-Accelerated Cryptography

2.3.1 Principles and Trade-offs of Software Cryptography

Software-based cryptography offers maximum flexibility and ease of deployment but places a significant burden on the limited computational and memory resources of IoT microcontrollers. Software implementations on general-purpose microcontrollers using lightweight libraries like mbedTLS or TinyCrypt maximize flexibility, supporting over-the-air patches and algorithm agility without hardware redesign [Faisal Abdullah Althobaiti, 2025]. However, these implementations demand higher CPU load and RAM, which increases latency in real-time control loops and rapidly depletes battery reserves [Faisal Abdullah Althobaiti, 2025]. For example, standard symmetric suites

introduce severe instruction cycle overhead when software-emulated on low-power microcontrollers lacking native hardware vector units [Ridho Rabbani, 2025].

As [Ridho Rabbani, 2025] said major constraint in software cryptography research is that benchmarks often rely on optimized C/assembly libraries in isolation, failing to account for the additional overhead induced by the overall software stack or the specific compiler configurations used in production environments. This resource-intensive nature of software emulation drives the integration of dedicated hardware components to offload the cryptographic burden.

2.3.2 Principles and Trade-offs of Hardware Cryptographic Engines

Dedicated hardware cryptographic engines provide superior energy efficiency and speed but introduce a rigid architecture that cannot adapt to evolving security standards. Hardware accelerators, such as ASICs or dedicated coprocessors, can reduce cryptographic energy consumption per operation by 50% to 80% compared to software-only implementations [Faisal Abdullah Althobaiti, 2025]. Empirical tests on ARM Cortex-M3 platforms confirm that storing AES matrixes in hardware significantly reduces power consumption and calculation requirements [Hung & Hsu, 2018]. However, building these special-purpose modules requires massive initial investment, and unlike software, they are nearly impossible to update once deployed "in the wild," risking total obsolescence if a primitive is compromised or security parameters change [Devadas et al., 2022]. Much of the hardware-focused literature relies on post-synthesis simulations or high-end Field-Programmable Gate Array (FPGA) models that do not reflect the specific bus latencies and invocation costs encountered in mass-produced microcontrollers [Al-Shmailawi et al., 2026]. While highly efficient, the lack of flexibility in fixed silicon necessitates evaluating platforms that combine hardware acceleration with programmable cores, such as the ESP32.

2.3.3 The ESP32 Hardware Accelerator

The ESP32 platform incorporates native hardware peripherals to bridge the performance gap between time-tested security standards and the constraints of low-power IoT sensing. The ESP32 series features integrated hardware encryption accelerators that support AES, SHA-256, RSA, and RNG [Al-Mashhadani & Shujaa, 2022]. Leveraging these built-in coprocessors for AES operations enables the device to

maintain real-time performance with encryption latencies under 3 milliseconds while keeping CPU usage stable at 35–45% [Rafat et al., 2025]. This allows the platform to support advanced security protocols like TLS 1.2 and authenticated communication via ESP-NOW without the prohibitive power drain associated with software-emulated block ciphers [Bideh et al., 2020].

Authors acknowledge that if hardware peripherals are not explicitly invoked by the specific library version (e.g., certain builds of mbedTLS), the system may default to software emulation without alerting the developer [Ridho Rabbani, 2025]. The availability of both native hardware modules and the capacity for software-optimized ciphers on a single platform creates a conflict in the literature regarding the most efficient path forward. The accelerator covers the AES block operation; it does not remove the per-call cost of preparing the cipher context (key expansion and loading), which is examined in Chapters 4 and 5.

2.3.4 Comparative Literature: Software ASCON vs Hardware AES

Current research presents conflicting conclusions on whether purpose-built lightweight software standards or hardware-accelerated legacy algorithms provide the optimal security foundation for the IoT. Proponents of lightweight cryptography demonstrate that software-based ASCON-128 can cut execution latency by 74% compared to software-emulated AES-128 on identical ESP32 silicon [Ridho Rabbani, 2025]. Furthermore, ASCON variants often outperform software AES in throughput and energy efficiency across various Arduino-based 32-bit platforms [Tripathi et al., 2024]. Conversely, other literature maintains that the presence of dedicated hardware accelerators, such as those in the ESP32 or LAUNCHXL-CC1310, makes legacy standards like AES practical and far more efficient than any software-only alternative [Rafat et al., 2025; Hung & Hsu, 2018].

The literature is not unified on which approach is superior, and the disagreement is traceable to what is actually being compared rather than to a genuine conflict in evidence. Studies reporting ASCON's superiority [Ridho Rabbani, 2025] benchmark it against software-implemented AES, where ASCON's lower computational overhead gives it a clear advantage. Studies reporting hardware-accelerated AES's superiority [Hung & Hsu, 2018; Rafat et al., 2025], by contrast, compare it against the same

software AES baseline being offloaded to silicon not against ASCON at all. No reviewed study directly benchmarks software ASCON-128 against hardware-accelerated AES-128-GCM on the same platform. This means the "AES vs ASCON" debate and the "software vs hardware" debate have proceeded largely in parallel, and their intersection precisely the comparison this dissertation performs remains empirically untested. Resolving this requires examining how existing studies actually measure and compare these approaches and whether those measurements are even comparable to one another.

2.4 Evaluation Methodologies for Energy-Centric Benchmarking

2.4.1 Metrics and Benchmarking Practices

The absence of a standardized metric framework for evaluating cryptographic performance leads to fragmented data that prevents equitable comparisons between legacy and lightweight algorithms. Current literature utilizes a diverse array of performance indicators, including energy-per-encrypted-kilobyte, cycles-per-byte, and composite "target functions" that assign weighted values to CPU cycles, initialization time, and memory usage [Ibrahim et al., 2025]. To address this fragmentation, [Jin & Becker, 2022] established a foundational benchmarking approach specifically for the ESP32 platform, providing a structured framework for transforming raw hardware measurements into numerical evidence to accurately assess the relationship between cryptographic overhead and resource utilization. These comparisons are further complicated by inconsistent definitions of "code size," with some studies including decryption and key-scheduling routines while others measure encryption in isolation [Beaulieu et al., 2013]. For example, performance evaluations of the ASCON family often emphasize a balance between throughput and memory footprint, whereas AES benchmarks frequently prioritize raw execution speed without accounting for the RAM-to-energy trade-offs essential for constrained devices [Al-Shmailawi et al., 2026].

Researchers themselves acknowledge that differing assumptions; such as the exclusion of key management overheads or the use of different cell libraries; make direct "fair comparisons" across studies highly problematic [Beaulieu et al., 2013]. This methodological inconsistency explains why the AES-vs-ASCON debate remains

unresolved, as the reported "superiority" of an algorithm is often a byproduct of the specific metrics prioritized by the investigators.

2.4.2 Measurement Techniques and Instrumentation

Variation in measurement approaches from high-precision physical analyzers to software-based simulations introduces environmental factors and instrumentation noise that complicate accurate assessment of the true energy demands of IoT security. Measurement setups in the literature range from specialized side-channel evaluation boards (SASEBO) and high-cost industrial tools like Otii Arc to low-cost custom platforms such as EMPIOT [Dezfouli et al., 2018]. A significant portion of existing research continues to rely on analytical models or simulators (e.g., CupCarbon) that frequently omit critical hardware factors such as start-up energy costs, MAC-layer complexities, and voltage stabilization effects [Mahdi & Abdullah, 2025]. Differences in hardware fidelity imply that reported energy savings may not be replicated on physical ESP32 nodes, where factors such as battery discharge behavior and radio interference substantially shape the overall power profile [Mahdi & Abdullah, 2025].

The research community explicitly identifies that many existing tools suffer from limited current measurement ranges or low accuracy when capturing the rapid, high-power spikes characteristic of cryptographic bursts and wireless transmissions [Dezfouli et al., 2018]. Recent methodological advancements by [Mitchel et al., 2025] mitigate these accuracy concerns by standardizing high-precision timing through the `micros()` API and defining rigorous throughput formulas designed to isolate computational latency from serial communication noise. Furthermore, their work introduces essential statistical treatments for cleaning raw logs, providing a structured process to remove experimental noise and mitigate the impact of random spikes or background system processes.

A further practical constraint concerns temporal resolution. Low-cost current monitors such as the INA219 convert the shunt voltage at a fixed rate: at 12-bit resolution one conversion takes about 532 μs , and the device offers hardware averaging over up to 128 samples (about 68 ms per result) [Texas Instruments, 2008]. A cryptographic call that lasts a few milliseconds, or less, therefore cannot be resolved by two instantaneous register reads taken before and after it, because each register holds the result of the most recently completed conversion rather than the instantaneous current. The usual remedy

is to repeat the operation continuously for long enough that several full conversions fall inside the measurement window, and to derive energy per operation from mean power and time per operation. Studies that report per-operation energy for millisecond-scale operations should therefore state how the sensor's conversion time was accommodated.

These inconsistencies become especially consequential once real-world duty-cycle behavior is considered, not just isolated encryption operations.

2.5 Realistic IoT Duty Cycles and Energy Implications

The energy profile of a field-deployed IoT node is primarily defined by its duty cycle the ratio of time spent in low-power sleep states versus active processing rather than by cryptographic operations in isolation. In realistic scenarios, such as environmental sensing or precision agriculture, devices operate in a cyclic pattern: transitioning from deep sleep to a brief wake-up phase for sensing and data processing before transmitting results and returning to sleep [Gerndt et al., 2025]. Research indicates that for nodes aiming for multi-year lifespans on coin-cell batteries, the duty cycle is typically kept very low, often under 13%, meaning the device spends over 85% of its existence in a dormant state [Aydin et al., 2011]. Consequently, the total energy budget is dominated by the cumulative cost of these sleep intervals and the significant transient energy spikes incurred during the state transitions between sleep and active modes [Dezfouli et al., 2018].

A significant methodological gap exists where duty-cycle models frequently omit the "start-up energy cost" the power consumed as voltage stabilizes and peripherals initialize which can be higher than the energy required for actual data transmission [Junchao Ma, 2009]. This suggests that lab-bench benchmarks focusing on single, continuous encryption operations fail to account for the transient overheads that define the true energy landscape of a deployed IoT node.

2.5.1 Duty Cycle Structure (Sleep → Wake → Sense → Transmit → Sleep)

The discrete phases of an IoT duty cycle introduce diverse power demands that must be managed collectively to ensure system longevity. The cycle begins with a wake-up trigger either an internal timer or an external interrupt followed by a "transient" startup

process where the radio and oscillator initialize, typically consuming around 22 μJ over 2.1 ms [Nkemeni et al., 2023]. Once active, the node performs sensing and computation, such as running a Kalman filter, where current draw on an ESP32 can jump from 300 μA in sleep to over 36 mA [Nkemeni et al., 2023]. The final active phase involves radio transmission, which represents the most power-intensive burst, before the node executes a "turndown" sequence to re-enter a sleep state [Junchao Ma, 2009].

Most experimental evaluations of these cycles are validated only via simulations, with very few studies performing physical measurements on laboratory testbeds to verify how specific application-layer tasks, like encryption, impact the total cycle duration [Nkemeni et al., 2023]. Understanding these distinct phases reveals that encryption is merely one component of a broader energy chain, necessitating its evaluation within the context of the entire active window.

2.5.2 Sleep Implementation Mechanisms

Advanced sleep modes, particularly deep sleep and hibernation, are essential for reducing the baseline current consumption of IoT platforms during periods of inactivity. The ESP32 provides several power-saving tiers, including modem sleep, light sleep, and deep sleep, with the latter reducing current consumption to as low as 10 μA –150 μA by shutting down the CPU, Wi-Fi, and Bluetooth [Mac et al., n.d.]. In deep sleep, the only active components are the Real-Time Clock (RTC) and the Ultra-Low Power (ULP) coprocessor, which can be programmed to monitor sensors and trigger a full system wake-up only when specific thresholds are met [Syahmi Md Dzahir & Seng Chia, 2023]. Utilizing these mechanisms has been shown to improve battery runtime by over 500% compared to systems that remain in a continuous idle state [Syahmi Md Dzahir & Seng Chia, 2023].

A critical technical constraint of deep sleep is that RAM contents are lost upon entry, forcing the microcontroller to undergo a full reboot and potentially costly algorithm re-initialization whenever it wakes up [Gerndt et al., 2025]. The necessity of re-initializing security parameters after every sleep cycle highlights a hidden energy cost that is often ignored in standard cryptographic benchmarks.

2.6 Synthesis of Comparative Evidence

The current body of literature presents a fragmented landscape where the perceived superiority of a cryptographic approach depends largely on the level of analysis and the conditions of implementation. At the algorithmic level, purpose-built lightweight standards like ASCON-128 are consistently championed for their streamlined sponge constructions and bit-sliced designs, which are claimed to bridge the "security-performance gap" that legacy block ciphers cannot meet [Ridho Rabbani, 2025]. However, this algorithm-centric view frequently conflicts with implementation-level research, which demonstrates that the energy and latency penalties of the Advanced Encryption Standard (AES) are effectively neutralized when offloaded to dedicated hardware accelerators [Hung & Hsu, 2018].

This fundamental conflict remains unresolved because the literature lacks a uniform methodological foundation. The disagreement over whether software-based lightweight standards or hardware-fixed legacy standards are more sustainable is often a byproduct of inconsistent metrics and disparate instrumentation. While some studies prioritize raw execution latency on 32-bit platforms, others focus on gate counts in ASIC simulations or energy-per-byte in isolated software libraries [Dezfouli et al., 2018]. Furthermore, the reliance on software-only baselines in ASCON performance profiles often ignores the "X-factor" of native silicon acceleration available on mass-produced platforms like the ESP32, leading to a research landscape where the two most viable paths for IoT security are rarely measured on an equitable field [Ridho Rabbani, 2025].

Finally, a critical disconnect persists between cryptographic benchmarking and the operational reality of the Internet of Things. Nearly all reviewed studies evaluate encryption as a continuous, isolated operation, failing to situate these results within a realistic hardware duty cycle. By ignoring the transient energy spikes of wake-up phases, the "start-up cost" of peripherals, and the cumulative impact of sleep-state transitions, existing research provides an incomplete picture of how algorithm choice actually influences the total energy budget and longevity of a battery-powered node [Nkemeni et al., 2023].

2.7 Research Gap and Justification

Despite the volume of research into lightweight cryptography and hardware acceleration, a significant empirical gap remains: no reviewed study performs a direct, head-to-head comparison between software-based ASCON-128 and hardware-accelerated AES-128-GCM under identical, realistic IoT duty-cycle conditions using a uniform energy-per-byte metric. While proponents of ASCON benchmark it against software-emulated AES to prove its speed, and hardware researchers benchmark silicon-based AES against software-based AES to prove its efficiency, the intersection of these two domains comparing the new software standard against the legacy hardware standard on the same platform remains empirically untested [Ridho Rabbani, 2025]. In addition, none of the reviewed studies reports the fixed cost of preparing the hardware cipher context separately from the cost that scales with payload size, although that split determines at which message size a hardware accelerator becomes worthwhile.

This gap is significant because the absence of comparative data forces developers into a dangerous trade-off between security posture and device sustainability. If the energy overhead of legacy standards is overestimated, developers may disable security features entirely to preserve battery life, leaving devices vulnerable to botnet-driven DDoS attacks and firmware exploits. Conversely, if a "lightweight" software standard is adopted without accounting for its behavior during rapid sleep-wake transitions, it may inadvertently deplete energy reserves faster than a hardware-fixed alternative, causing premature system failure in inaccessible or hostile environments where battery replacement is impractical. Resolving this contradiction is essential for establishing a sustainable security foundation that can extend theoretical device lifetimes by years while maintaining robust, standardized protection for the global IoT ecosystem.

Chapter 03: System Design and Methodology

3.1 Research Process

This study adopts a quantitative research framework grounded in a positivist paradigm, assuming that performance metrics are objective realities that can be empirically verified through structured experimentations. This dissertation prioritized the collection of numerical evidence, provide statistically significant insights into the behavior of the cryptographic algorithms under study. The approach of [Jin & Becker, 2022] facilitates the transformation of raw hardware measurements into structured analytical tools, such as comparative tables and performance graphs, to accurately assess the relationship between cryptographic overhead and resource utilization. An experimental strategy is utilized to collect data in standardized units of measurement, allowing pre-defined framework to interpret data, rather than building theory from observations.

3.1.1 Research Methodology

The methodology developed for this research significantly extends the foundational benchmarking approach established by [Jin & Becker, 2022], which primarily focused on execution time and throughput. While Jin and Becker's work provided critical baseline performance data, this study expands that scope by introducing power consumption and realistic duty cycle dimensions to assess the total energy impact of cryptographic operations. Furthermore, this dissertation explicitly addresses the limitations identified in recent literature by [Amirkhanova et al., 2026], which noted that many prior end-to-end pipelines fails to quantify energy utilization or CPU overhead beyond basic latency. The investigation is specifically designed to evaluate the performance of hardware-accelerated AES-128-GCM against the software-based ASCON-128 standard. These algorithms are assessed using a comprehensive set of metrics, including throughput, memory allocation, and energy cost per transmitted byte. To ensure the findings reflect actual IoT operational conditions, these variables are measured across the full lifecycle consisting of sleep, wake-up, sensing, encryption, and transmission phases. To address the identified gap, this investigation incorporates per-byte energy efficiency analysis, normalizing power consumption across varying payload sizes.

3.2 Selection of Cryptographic Algorithms

The selection of cryptographic algorithms for this evaluation is designed to compare the established global "gold standard" against the emerging standard for lightweight security in the Internet of Things (IoT). For this purpose, AES-128-GCM and ASCON-128 were chosen as the two primary candidates [Amirkhanova et al., 2026; Fernández-García & Jiménez-Fernández, n.d.]. This selection allows the research to analyze the performance gap between a traditional block cipher that benefits from native ESP32 hardware acceleration and a modern lightweight cipher optimized for efficiency in software-only environments.

3.2.1 Rationale for AES-128-GCM

AES-128 was retained as the symmetric baseline for this study to ensure comparability with a vast body of prior literature evaluating the ESP32's hardware-acceleration capabilities. However, this research transitions from commonly used Cipher Block Chaining (CBC) mode to Galois Counter Mode (GCM). The primary rationale for choosing GCM is that it provides Authenticated Encryption with Associated Data (AEAD), which integrates confidentiality and message integrity into a single cryptographic pass. By utilizing AES-128-GCM, the study can measure the maximum efficiency of the ESP32's dedicated hardware engine when performing the authenticated encryption tasks required for secure industrial communication.

3.2.2 Rationale for ASCON-128

ASCON-128 is implemented as the primary lightweight comparison point against the hardware-accelerated AES baseline. Selected by NIST in 2023 as the global standard for lightweight cryptography, ASCON is explicitly designed for resource-constrained devices where traditional algorithms may be too compute-intensive [Fernández-García & Jiménez-Fernández, 2024]. Unlike AES, the ESP32 does not possess a dedicated hardware accelerator for ASCON, which allows this research to isolate the performance impact of hardware acceleration versus a purely software-optimized lightweight cipher. This comparison is vital to determine whether the algorithmic efficiency of ASCON in software can match or exceed the energy-saving benefits of legacy hardware-offloading for AES.

3.2.3 Common Evaluation Basis

To strengthen the methodological rigor of the comparative evaluation, a controlled-variable design is utilized. All environmental factors, including payload generation, the timing method, and the duty cycle, are held constant across both AES and ASCON implementations.

The table below summarizes the evaluation methods for each metric.

Metric	Description	Measurement Method/ Tool
Execution Time	The total duration required to complete a single cryptographic operation (encryption or decryption).	Measured using the built-in <code>esp_timer_get_time()</code> function, capturing microsecond-resolution snapshots before and after each operation.
Throughput	The volume of data processed per unit of time, expressed in bytes per second (B/s).	Calculated using the formula: Throughput = Bytes in payload / mean execution time(s)
Power Consumption	The real-time electrical energy drawdown during active cryptographic processing.	Monitored via an INA219 current sensor, allowing for the tracking of milliampere-hours (mAh) or microjoules (μJ) per operation.
Memory Footprint	The impact of the cryptographic algorithm on the device's volatile memory (RAM).	Captured using the <code>heap_caps_get_free_size()</code> function to measure the difference in available heap memory before and after the process.

Table 2 - Summary of Evaluation Metrics and Methodologies

The evaluation is conducted within the ESP-IDF (Espressif IoT Development Framework), the official software development environment for the ESP32 SoC, which provides the necessary libraries and low-level hardware access for accurate benchmarking [Al-Mashhadani & Shujaa, 2022].

3.3 Experimental Design

The experimental design is structured to facilitate a granular, energy-centric comparison between hardware-accelerated and software-based cryptographic primitives. The framework integrates performance benchmarking with power profiling across a realistic IoT lifecycle, ensuring that the results reflect actual operational conditions rather than continuous, high-power execution.

3.3.1 Overview of the Measurement Program

The measurement program follows a structured logic to capture the computational and energy overhead of cryptographic operations on the ESP32 platform. As illustrated in the Main Setup and Nested Loop Control flowchart, the process begins by selecting the encryption algorithm (hardware-accelerated AES-128-GCM or software-based ASCON-128) and defining the corresponding cryptographic key. Once a stable Wi-Fi connection is established for data reporting, the system enters a dual-nested loop structure designed for high statistical granularity.

- **Outer Loop ():** Executes 500 iterations per data point to ensure statistical stability and mitigate the impact of transient system noise.
- **Inner Loop ():** Iterates through 100 varying payload sizes (where ranges from 1 to 100), enabling the study to evaluate performance scaling across a broad range of data volumes.

This program is referred to as Campaign A. A second, sustained-load program (Campaign B) was developed to validate its energy measurements and is described in 5.3.

Each iteration of the inner loop triggers a single RUN_TRIAL sequence. As defined in the Payload Generation algorithm, the system dynamically creates a message where the $\text{Size} = 16 * j$ resulting in payloads ranging from 16 bytes to 1,600 bytes. To ensure reproducibility, these payloads are generated using a Pseudo-random Number Generator (PRNG) seeded with a constant value of 128.

Prior to encryption, a unique nonce is generated for every message to maintain cryptographic integrity. According to the GENERATE_NONCE flowchart, the system applies algorithm-specific lengths: a 12-byte nonce for AES-128-GCM, consistent with GCM standards, and a 16-byte nonce for ASCON-128.

ENCRYPT_AND_MEASURE function capture core performance data. This activates a power and memory monitor immediately before the encryption function and stops it precisely upon cipher termination. To quantify the total energy cost of security within a realistic IoT duty cycle, the program executes a SLEEP_AND_MEASURE routine following each encryption trial. This process encompasses the following stages:

1. **Baseline Monitoring:** Power monitoring is initialized to capture the sleep-state draw
2. **Deep Sleep:** The ESP32 enters a 60-second-deep sleep phase, significantly reducing current consumption by powering down the CPU and radio
3. **Wake-up and Reconnection:** Following the sleep interval, the device wakes up and executes a RECONNECT_WIFI() command

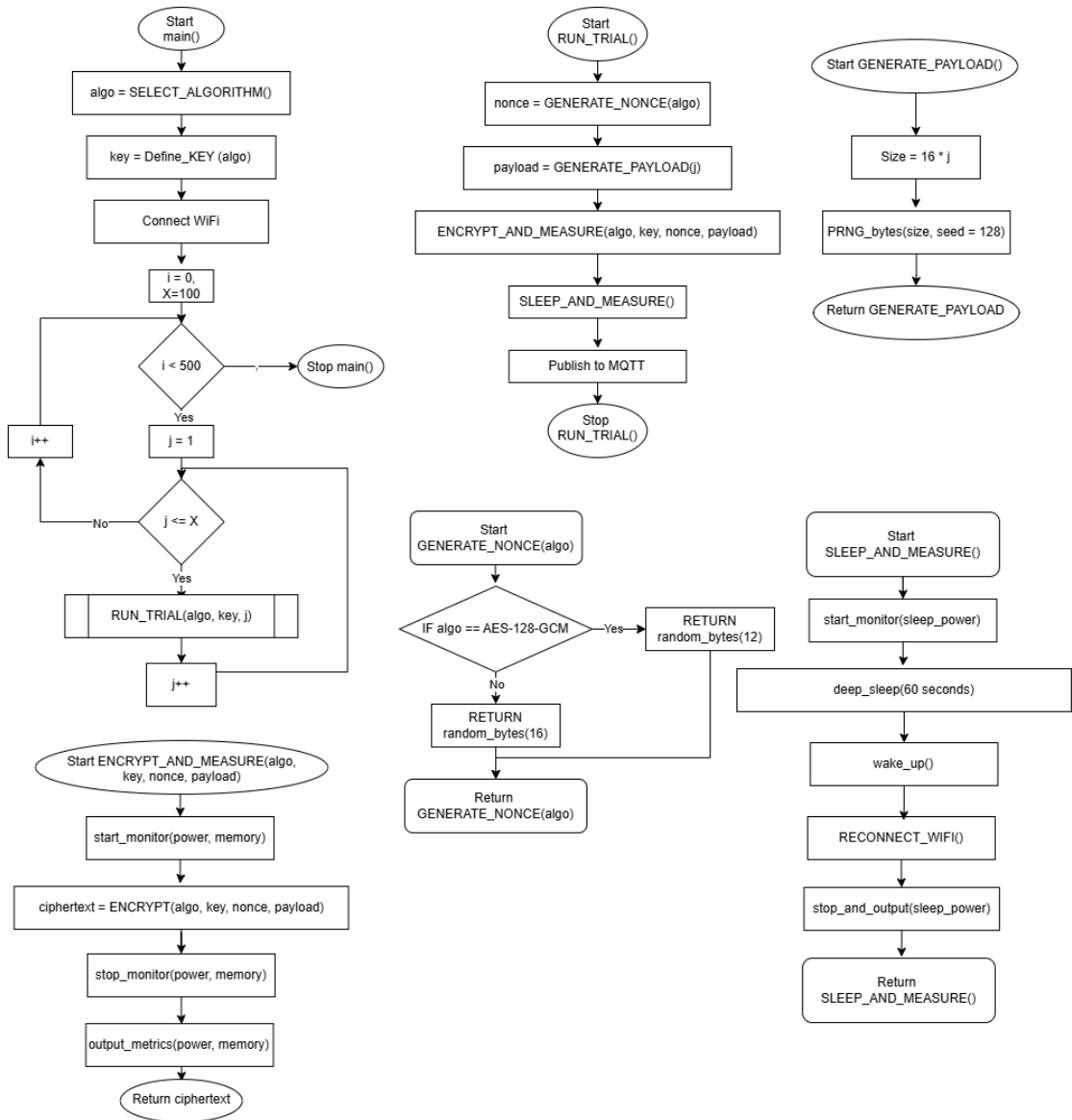


Figure 1 - Main Algorithm

4. **Reporting:** Once the connection is restored, the accumulated metrics are transmitted to a remote Node-RED dashboard for real-time visualization and logging using MQTT protocol and logged into text files.

This integrated approach allows the study to measure the complete energy chain from the initial wake-up and sensing through encryption to the post-sleep reporting providing a comprehensive view of the security-to-battery life trade-off.

3.3.2 Payload Generation Strategy

The payload generation strategy utilizes a benchmarking approach established by [Jin & Becker, 2022] to assess how varying data volumes affect cryptographic resource consumption. The measurement program evaluates 100 specific data points (), starting from a baseline of 16 bytes and incrementing by 16 bytes per step until reaching a maximum payload of 1600 bytes. Each data point is executed and measured 500 times to minimize measurement uncertainty and provide a representative mean latency and power draw.

To ensure reproducibility across experimental runs, a Pseudo-random Number Generator (PRNG) is used, seeded with a constant value of 128. This generator builds data messages where each byte is randomly selected from a pool of numerical characters (0-9) and lowercase alphabets (a-z). This method eliminates the risk that specific, non-random data patterns might artificially bias algorithm performance.

3.3.3 Nonce Generation

The security of both AES-128-GCM and ASCON-128 depends critically on nonce uniqueness to prevent replay attacks and ensure cryptographic integrity. Following the research algorithm, a fresh nonce is generated for every individual message prior to the encryption phase. For the AES-GCM implementation, a 12-byte nonce is generated using the ESP32's internal cryptographic randomness, consistent with established security standards for Galois Counter Mode. And 16-byte nonce is for ASCON-128 is used as well. This nonce is then cryptographically bound to the resulting ciphertext and subsequently shared during the transmission phase for successful decryption at the gateway.

3.3.4 Duty-Cycle/ Sleep Design

The duty-cycle design is modeled after the environmental monitoring system described by [Syahmi Md Dzahir & Seng Chia, 2023] to simulate realistic IoT behavior. Rather than operating in a continuous active state, the ESP32 is configured to follow a Sleep → Wake → Sense → Encrypt → Transmit → Sleep cycle. The reporting intervals

modelled in this dissertation is 60 seconds. The sleep phase was not measured: its energy is added analytically (4.2.4 and 5.4.8). In accordance with Syahmi's methodology, the device enters a deep sleep mode for 60 seconds between each active session to maximize battery longevity. It should be noted that while Syahmi's original work investigates energy consumption across four distinct conditions by varying both FreeRTOS (on/off) and deep sleep (on/off) states, this research exclusively adopts the deep-sleep dimension of their design, utilizing a bare-metal implementation to isolate cryptographic energy overhead from operating system scheduling noise.

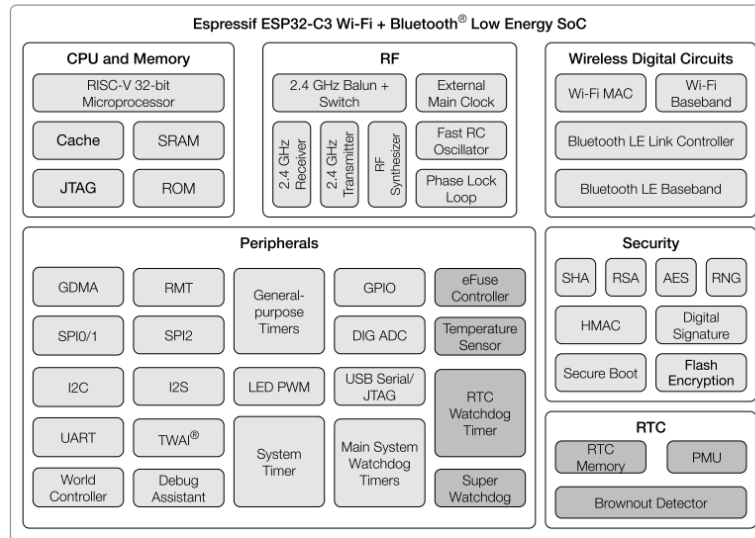
During the active transmission phase, the device utilizes Wi-Fi and the MQTT protocol to share the ciphertext, secret key, and generated nonce with a Node-RED dashboard. This step implements the transmission dimension of the duty cycle, allowing for real-time visualization of the cryptographic data while capturing the significant power draw associated with radio-frequency (RF) operations. By measuring energy across this complete cycle, the research can accurately identify the break-even payload size where hardware-acceleration may become more efficient than lightweight software alternatives.

3.4 Test-bed Implementation

The implementation of the research test-bed is designed to provide a high-precision environment for measuring the cryptographic performance and energy efficiency of the ESP32 platform. This setup integrates standard IoT telemetry components with dedicated power monitoring hardware to capture granular data across the complete duty cycle.

3.4.1 Hardware Components

The hardware setup is designed to provide an accurate and stable environment for measuring the cryptographic overhead of the ESP32-WROOM-32E development board, while the INA219 sensor captures energy usage data with high precision.



Power consumption

Figure 2 - Functional Block Diagram of ESP32

1. ESP32-WROOM-32E Microcontroller
The heart of the test-bed is the ESP32-WROOM-32E a highly integrated System-on-Chip (SoC) designed for low-power IoT applications. It features a dual-core

Xtensa 32-bit LX6 microprocessor capable of operating at clock frequencies up to 160MHz, providing ample computational power for handling complex cryptographic tasks in both software and hardware implementations (Mac et al., n.d.).

- **Memory and Peripherals:** The SoC includes approximately 400 KB of internal SRAM for data and instruction storage and supports external flash memory up to 16 MB. Its integrated Wi-Fi and dual-mode Bluetooth connectivity allow it to function as a standalone MQTT client for real-time telemetry reporting.
- **Cryptographic Hardware Acceleration Engine:** The ESP32 is distinguished by its dedicated security subsystem, which includes hardware accelerators for widely used algorithms such as AES (128, 192, and 256-bit), SHA, and RSA. This engine works by offloading computationally intensive mathematical operations from the main CPU to specialized silicon blocks. This offloading drastically reduces encryption latency and total energy consumption, as a hardware-based AES operation can execute in a fraction of the time required by a software library. The accelerator can operate in Typical Mode, where the CPU interacts directly with registers, or DMA

Mode, which utilizes the General DMA (GDMA) controller to transfer data between RAM and peripherals without continuous CPU intervention.

- **Power Management and Deep Sleep:** To maximize battery life in realistic IoT duty cycles, the ESP32 utilizes a multi-mode power management unit. The most efficient mode is Deep Sleep, where the CPU, Wi-Fi, and Bluetooth are powered down, leaving only the Ultra-Low-Power (ULP) co-processor or the Real-Time Clock (RTC) active. In this state, the device consumes as little as 5 μA to 10 μA , significantly extending operational longevity compared to continuous active operation. The device can be woken up via a periodic RTC timer or external interrupts, at which point it returns to the active mode to sense and encrypt data.

2. INA219 Current and Power Monitor

To monitor the energy efficiency of the cryptographic operations, an INA219 current sensor is integrated into the circuit. This device is a high-side, bidirectional current shunt and power monitor with an I2C-compatible interface [Texas Instruments, 2008].

- **Operating Mechanism:** The INA219 functions by sensing the small voltage drop across a shunt resistor (R_{SHUNT}) placed in series with the load (the ESP32). It simultaneously measures the bus supply voltage (up to 26V) and the differential shunt voltage.
- **Data Processing:** The sensor features a programmable internal multiplier and calibration registers that allow it to calculate current in amperes and power in watts directly. This data is digitized by an internal 12-bit ADC, allowing the researcher to retrieve granular power consumption metrics via

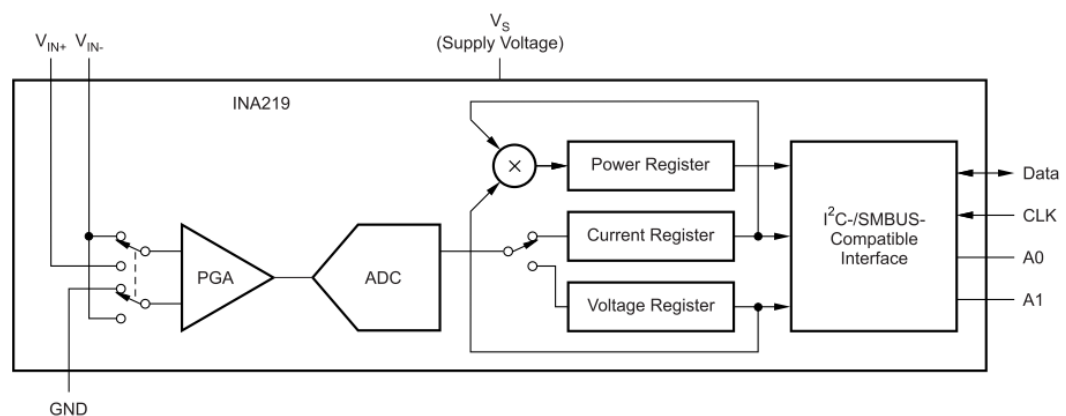


Figure 3 - Simplified Schematic of INA219

the ESP32's I²C bus at GPIO 21 (SDA) and GPIO 22 (SCL). This setup provides the microsecond-level precision necessary to capture the transient power spikes associated with hardware-accelerated encryption tasks.

3. Supporting Infrastructure

- **Power Supply:** The assembly is powered by a regulated 5V DC source. This voltage is stepped down to the 3.3V operating level required by the ESP32 via a buck convertor, ensuring a stable baseline for all current measurements.

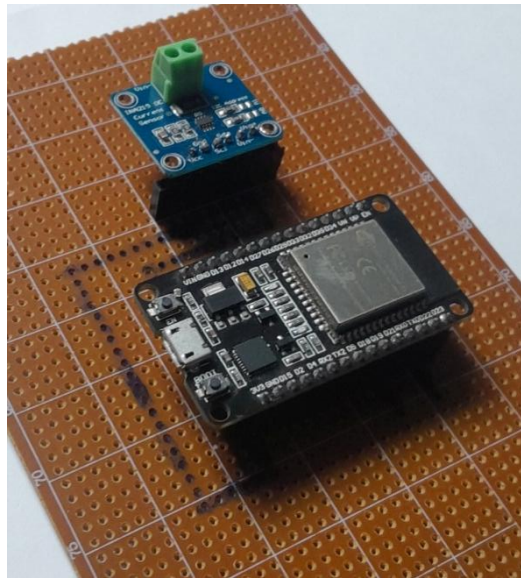


Figure 4 - Test Bed Side View

3.4.2 Software Environment and Libraries

The evaluation is conducted within the ESP-IDF (Espressif IoT Development Framework), the official software development environment for the ESP32 SoC. This framework provides the necessary APIs, scripts, and an integrated development environment (IDE) for developing programs in the ANSI C standard using the gcc compiler. Key software components and libraries utilized include:

Cryptographic Libraries: The mbed TLS library (e.g., version 2.16.2) is employed to handle transport layer security and provide the interface for AES hardware acceleration, which is a native feature of the ESP32. It is critical that the implementation explicitly invokes these hardware peripherals to ensure the system does not default to slower software emulation.

Communication Protocols: Data transmission is managed through the ESP-IDF MQTT library (running version 3.1.1), while the Mosquitto broker (e.g., version 1.6.7) is utilized on the server side to facilitate message distribution.

3.4.3 Schematic Diagram

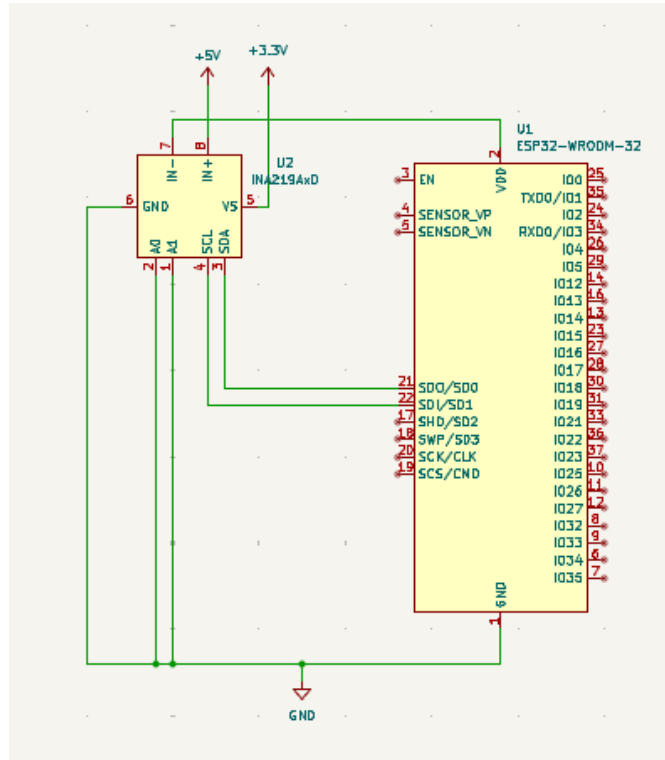


Figure 5 - Schematic Diagram

The Fig 5 schematic diagram illustrates the precise electrical interconnects required to facilitate granular energy profiling of the ESP32-WROOM-32 module using the INA219 current sensor. The design ensures that the cryptographic operations performed by the microcontroller are accurately captured electrical data.

- The circuit is designed for high-side current sensing, where the INA219 is placed between the power source and the load (the ESP32). The +5V supply rail is connected directly to the IN+ (Pin 8) of the INA219.
- The load output N- (Pin 7) of the INA219 is connected to the VDD (Pin 2) of the ESP32. This configuration allows the INA219 to measure the differential voltage drop across its internal shunt resistor, which converted into current and power reading.

- To retrieve measurement data, ESP32 functioned as I2C master, communicating with the INA219 as a slave. The SDA (Pin 3) and SCL (Pin 4) pins of the INA219 are interfaced with the ESP32's GPIO 21 and GPIO 22, respectively.
- The address pins A0 (Pin 2) and A1 (Pin 1) on the INA219 are both tied to GND. According to the INA219 datasheet, this hard-wired configuration sets the unique I2C slave address to 1000000 (0x40), allowing the ESP32 to target the sensor on the bus.

This schematic implementation is critical for the deductive analysis of the research. By measuring the voltage at IN- and the differential voltage between IN+ and IN-, the ESP32 can calculate its own instantaneous power draw during the "Encrypt" and "Transmit" phases of the duty cycle. This setup provides the microsecond-level precision necessary to distinguish between the energy profiles of software-based ASCON-128 and hardware-accelerated AES-128-GCM.

3.4.4 Wiring Diagram and Physical Setup

The physical implementation of the test-bed transitions from a temporary prototype to a stable, soldered assembly on a prototyping dot board, as illustrated in the provided hardware documentation. This implementation ensures maximum electrical integrity and minimizes parasitic contact resistance. This permanent setup is critical for the research's energy-centric evaluation, as it eliminates the risk of power fluctuations or voltages drops cause by loose physical connections.

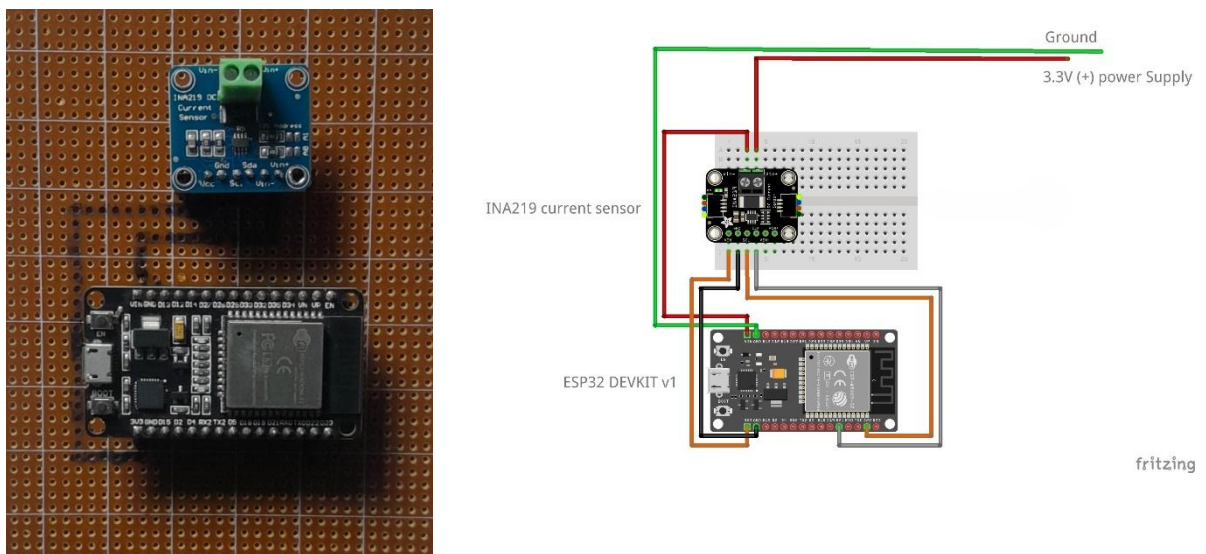


Figure 6 - Test Bed

As the Fig.6 depicts the assembly features ESP32-WROOM-32E and INA219 sensor mounted securely on the board, with the sensor positioned in series with the 5v supply rail to capture the differential voltage drop across the internal shunt. The communication lines for the I2C interface (SDA and SCL) are hard-wired between the components, ensuring a reliable data path for real-time power reporting.

3.5 Measurement Methodology

3.5.1 Execution Time Measurement

Execution time is measured using high-resolution hardware timers to isolate the computational latency of the cryptographic primitives from background operating system tasks. Following the methodology established by [Mitchel et al., 2025], the research utilizes the `micros()` clock API, which provides a precision of one microsecond.

As defined in the research algorithm, the measurement program records a high-precision timestamp immediately before the cryptographic function call and another immediately after the operation concludes. The difference between these two values represents the total latency for that specific payload. To ensure statistical reliability and account for transient system fluctuations, this process is integrated into a nested loop structure: an inner loop `()` iterates through 100 varying payload sizes, while an outer loop `()` repeats each test 500 times to establish a stable mean latency for every data point.

3.5.2 Throughput Calculation

Throughput is calculated to quantify the data processing speed of each algorithm, representing the volume of data encrypted or decrypted per unit of time. The calculation follows the standardized mathematical formula utilized in the [Mitchel et al., 2025] research.

The payload size for each calculation is derived from the generation strategy where the data message is defined as $16 \text{ bytes} \times j$. Once the 500 execution time samples (in microseconds) are collected for a specific payload size, the researcher calculates the mean time per operation (x). This mean value is then converted into seconds to determine the throughput in bytes per second (B/s) using the following formula:

$$\text{Throughput(B/s)} = \frac{(16 \times j) \text{ Bytes}}{\text{Mean Execution Time}}$$

Equation 1 - Throughput Equation

By applying this calculation across the entire range of the payload generator (16 to 1600 bytes), this metric allows for a direct, normalized comparison of how effectively the ESP32 hardware engine offloads processing compared to the CPU-bound ASCON-128 software implementation.

3.5.3 Power Consumption Measurement

Power consumption is monitored using the INA219 current sensor, integrated directly into the ESP32 circuit to capture real-time electrical data. This high-precision sensing approach, as utilized by [Rafat et al., 2025] allows for the detection of transient current spikes during active cryptographic tasks.

The total system power is calculated based on the generalized power law, where electrical power () is the product of the constant supply voltage () and the instantaneous current () measured by the INA219.

$$P = V \times I$$

Equation 2 - Total Power Equation

The bus voltage measured on the 5 V rail was about 4.72 V. In Campaign A, power was estimated from two instantaneous readings around each cipher call; the limits of that method are examined in 5.2. In Campaign B, the extra power drawn by the cipher was measured as the difference between a sustained-load window and idle windows (5.3.2).

3.5.4 Memory Footprint Measurement

Memory footprint measured in Campaign A. Heap use was taken from ‘heap_caps_get_free_size()’ before and after the cipher call, and peak heap use from the change in ‘heap_caps_get_minimum_free_size()’. Stack use was taken from ‘uxTaskGetStackHighWaterMark()’ in a dedicated task created for each chipper call, so that stack use by other work does not contaminate the reading. The footprint is the sum of peak and stack use, averaged over the hundreds of sweeps collected. The technique follows the idea of before-and-after memory snapshots used by [Matas Pipiras, 2024],

implemented in ESP-IDF using VS code extension. It measures run-time memory only, not code size in flash.

3.6 Data Collection and Extraction

For the data collection and extraction phase, the methodology follows a structured process of capturing real-time metrics and logging them for evaluations later. Following the execution of each cryptographic operation, performance metrics specifically execution time and memory usage are generated by the microcontroller and transmitted to an external listener. In studies involving ESP32-based platforms, these results are typically published via protocols like MQTT or printed directly to a serial console. In this study, these metrics are outputted to the ESP-IDF console, where the raw data is captured and saved into a text file.

Once the iterations are complete, the saved output undergoes a data extraction and cleaning process to ensure accuracy. Similar to the analysis techniques used by [Matas Pipiras, 2024; Mitchel et al., 2025], the raw logs are processed to remove noise and formatted into structured files, such as CSV, for easier handling. This structured data is then used to perform statistical analysis, which includes calculating the median or mean values to mitigate the impact of random spikes or background system processes. Finally, the cleaned data is visualized through tables and graphs to identify performance trends across different encryption modes and payload sizes.

3.7 Expected Data Analysis and Statistical Evaluation

3.7.1 Statistical Methods and Tools

The data analysis phase involves processing raw performance metrics captured during the experimental trials. Initial data collection results in raw logs containing execution times and memory snapshots, which are subsequently cleaned and structured into CSV files for systematic evaluation. To ensure the reliability of the results and mitigate the impact of random noise, such as background system processes or network activity, each test configuration is subjected to multiple iterations.

- The median value is utilized to represent typical performance, as it effectively reduces the skewing effect of irregular spikes in execution time. For large-scale datasets involving 1000+ iterations, the mean execution time and throughput

(Bytes per second) are also calculated to provide a comprehensive view of performance.

- Standard deviation and 95% confidence intervals are computed to quantify the variability between iterations.
- The statistical analysis is facilitated by the Python library SciPy for evaluating p-values and cleaning raw serial communication data [Mitchel et al., 2025].

3.7.2 Significance/Comparison Method

To evaluate the performance trade-offs between different cryptographic configurations, the study employs several comparative techniques:

Hypothesis Testing: per-size difference between the ciphers were tested with two-sided t-tests (Welch’s test for the independent Campaign A samples; a paired test on per-replicate differences for Campaign B), because the direction of the difference reverses across the break-even. Because one test is run for each payload size, the p-values were adjusted for the family of tests with the Holm procedure (family-wise $\alpha = 0.01$), and effect sizes are reported next to significance. The break-even is the interpolated sign change of the mean difference, with a bootstrap 95% interval. An earlier analysis of Campaign A used one-tailed uncorrected tests; it is re-examined in 5.5.6.

Regression and Scaling Analysis: For metrics that change based on data size, such as RAM usage or associated data (AD) runtime, the study utilizes linear regression. The coefficient of determination (R^2) is used to confirm how closely the performance follows a linear growth pattern as payload sizes increase.

Data Visualization: Comparative findings are illustrated through bar charts for discrete category comparisons (like execution time across different AES modes) and scatter plots or line graphs to visualize scaling trends.

Chapter 04: Data Analysis and Results

4.1 Introduction

This chapter presented the results of the experimental evaluation described in chapter 3, comparing software-based ASCON-128 with hardware-accelerated AES-128-GCM on the ESP32-WROOM-32E across the full Sleep → Wake → Sense → Encrypt → Transmit → Sleep duty cycle. The chapter was structured to answer, in order, the three objectives set out in (1.8 objectives): the total duty-cycle energy and per-byte energy efficiency of each algorithm, including the hardware engine's initialization overhead (Objective 1); the energy cost per transmitted byte across the full duty cycle (Objective 2); and the conditions, including the break-even payload size, under which software-based ASCON-128 remained preferable to hardware-accelerated AES-128-GCM (Objective 3).

The measurements reported in this chapter were made with the per-trial benchmark described in 3.3.1, referred to as Campaign A. After a review, its measurements methods were audited and a second, sustained-load campaign (Campaign B) was carried out; both are reported in Chapter 5. The time increments and the memory footprint of this chapter were confirmed. The power, energy, break-even and significance results were changed, and chapter 5 states which findings stand, which are replaced and why.

The chapter began with a summary of how the raw benchmark logs were converted into the cleaned dataset that all subsequent statistics were computed from (4.2), since the reliability of every downstream result depended on that process. The remaining sections presented results by metric family: execution time and throughput (4.3), power consumption (4.4), per-byte energy and break-even analysis (4.5), memory footprint (4.6), and the statistical significance tests that supported the findings (4.7). 4.8 closed the chapter with a summary organized around the three objectives, setting up the interpretation carried out in chapter 5.

Both ASCON-128 and AES-128-GCM datasets were processed through an identical pipeline. The same iteration completeness filter, the same two sensor-calibration constants, and the same outlier-trimming procedure (4.2); Both datasets were processed through the same pipeline; the limits of the resulting measurements are assessed.

4.2 Data Collection Outcome and Cleaning Summary

4.2.1 Raw Log Volumes and Structural Completeness

The firmware described in 3.3 produced one newline-delimited JSON record per encryption trial, logged during the ‘ENCRYPT_AND_MEASURE’ phase of each duty cycle. The raw log for ASCON-128 contained 65,506 records and the raw log for AES-128-GCM contained 61,715 records; no malformed or truncated lines were encountered in either log during parsing.

Rather than de-duplicating these records by the trial “id” field, the logs were segmented into complete outer-loop iterations, since “id” was not guaranteed to increment independently across device resets. An “id” collision could span two genuinely different sweeps rather than indicate a retransmitted reading. An iteration boundary was instead detected from the payload-size sequence itself: because each sweep progressed through payload sizes from 16 bytes to 1,600 bytes in fixed 16-byte steps (matching “Size = 16 × j” from the payload generation algorithm in 3.3.2), a drop in the recorded size back to 16 bytes marked the start of a new sweep. This segmentation was performed on the log in its original recorded order.

The overall structure was an outer loop of repeated sweeps and an inner loop of 100 payload sizes stepping from 16 bytes to 1600 bytes in fixed 16-byte increments, with each message generated by a pseudo-random number generator seeded with a constant value of 128 following the benchmark design established for the ESP32 platform by (Jin & Becker, 2022) (3.3.2), whose own evolution used an identical outer-loop inner-loop structure (500 repeats x 100 payload sizes) but was limited to execution time and throughput. This study extended that design with the power, memory, and full-duty-cycle instrumentation absent from the original benchmarking approach, detailed in 4.2.2 – 4.2.4 below.

This process identified 699 candidate iterations in the ASCON-128 log and 646 candidate iterations in the AES-128-GCM log; more than the 500 outer-loop repeats specified in 3.3.1, since a number of these candidate sweeps were partial fragments left by device resets or reboots rather than genuine additional runs. Only iterations containing all the 100 distinct payload sizes exactly once were retained, partial sweeps were discarded rather than repaired or interpolated. This yielded 561 complete iterations for ASCON-128 payload levels, and 510 for every AES-128-GCM payload

level, with zero variance in the per-size sample count for either algorithm. Because both iterations exceed the predefined design, all complete samples were retained to analysis rather than subsampling to the fixed amount, in order to maximize statistical power.

Algorithm	Raw rows	Malformed lines	Candidate iteration	Complete iteration	Rows after iteration filter	Rows after outlier trimming
ASCON-128	65,506	0	699	561	56,100	45,745
AES-128-GCM	61,715	0	646	510	51,000	42,870

Table 3-Row counts per cleaning stage

4.2.2 INA219 Zero-Offset Calibration

Inspection of the iteration-complete data showed that approximately 21% ‘current_before’ and ‘current_after’ readings were negative. These negative readings clustered tightly between scattered measurement noise: the middle 50% of the negative readings clustered tightly between approximately -6.5mA and -5.2mA, with small number of much larger negative spikes (down to roughly -76mA and -99mA) occurring separately as genuine transient noise rather than part of the cluster. This tight, repeatable clustering was consistent with a sensor zero-offset error, in which the INA219’s calibrated “zero current” reading sits a constant amount below the true physical zero, shifting every reading, including physically valid positive ones-down by the same fixed amount.

Rather than marking these values to zero, which would discard the offset signal and bias every low-current reading by an inconsistent amount depending on its distance from zero, the offset was estimated directly from the data and corrected. Using the mean of the absolute median negative ‘current_before’ and ‘current_after’ values across the ASCON-128 iteration-complete data, giving a shared calibration constant of 5.70mA. This constant was added back to every current reading in both datasets; not only the negative ones, and the small number of readings that remained negative after correction were clipped at zero. Power was recomputed from the corrected current using an empirically measured bus voltage, rather than the device’s own power fields, which had inherited the same sensor bias. The bus voltage was estimated as the median of ‘power / current’ on clearly-active, high current rows, giving 4.674 V, lower than the nominal

5V supply rail, consistent with the voltage drop across the INA219's internal shunt resistor.

4.2.3 Outlier Trimming

After the structural (4.2.1) and calibration (4.2.2) corrections were applied, IQR-based trimming ($k=1.5$) was applied to execution time and the corrected power delta, grouped by payload size. This ordering was calculated: applying outlier trimming before the iteration-completeness filter or the calibration correction would have risked treating structural artifacts (partial sweeps) or systematic sensor bias as if they were statistical noise, rather than correcting them at their source.

This step removed 10,355 rows from the ASCON-128 dataset (56,100 $\rightarrow$ 45,745 rows) and 8,130 rows from the AES-128-GCM dataset (51,000 $\rightarrow$ 42,870 rows). Trimming was tested for its effect on the conclusions (5.5.5): the break-even was 345.0, 347.4 and 346.8 bytes without trimming, with $k=3$ and with $k=1.5$.

4.2.4 Sleep-Phase Energy Constant

The firmware instrumented only the active 'ENCRYPT_AND_MEASURE' phase of the duty cycle; deep-sleep phase was not measured on-device. Sleep-phase energy was instead added analytically, following 3.3.4's specification of a 60-second deep-sleep interval, using the relation:

$$E_{\text{sleep}} = V_{\text{sleep}} \times I_{\text{sleep}} \times T_{\text{sleep}}$$

Equation 3-Sleep-phase energy constant

The deep-sleep current figure was taken from the ESP32 Datasheet, Version 2.3 [Mac et al., n.d.] , table 5-9 ("Current consumption in Low-power modes"), which specifies a typical deep-sleep current of **5 μA** for RTC timer + RTC memory wake configuration used in this study, measured at a 3.3V supply and 25 °C ambient temperature (5.6.1 of that datasheet). Using ' $V_{\text{sleep}} = 3.3\text{V}$ ', ' $I_{\text{sleep}} = 5 \mu\text{A}$ ', and the design-specified ' $T_{\text{sleep}} = 60\text{s}$ ' gave: $E_{\text{sleep}} = 3.3\text{V} \times 5 \mu\text{A} \times 60\text{s} = 990 \mu\text{J}$. This value was added to every payload size's active-phase energy to figure to produce the total duty-cycle energy reported in 4.5.

4.3 Execution Time and Throughput Results

4.3.1 Execution Time Scaling

Mean execution time was computed for each of the 100 payload sizes, for both algorithms, from the cleaned trial-level data using (4.2.3), together with the median, standard deviation, and 95% confidence interval (using the t-distribution). At the smallest payload size (16 bytes), AES-128-GCM required 1,740.8 μs (95% CI: 1,740.7–1,741.0 μs , $n = 482$). At the largest payload size (1,600 bytes), ASCON-128's mean execution time has risen to 3,266.5 μs – marginally higher than AES-128-GCM's 3,167.6 μs , despite AES-128-GCM being slower at every smaller payload size tested. Figure 7 presents the full execution-time curve for both algorithms, with the 95% confidence interval shown as a shaded band around each mean.

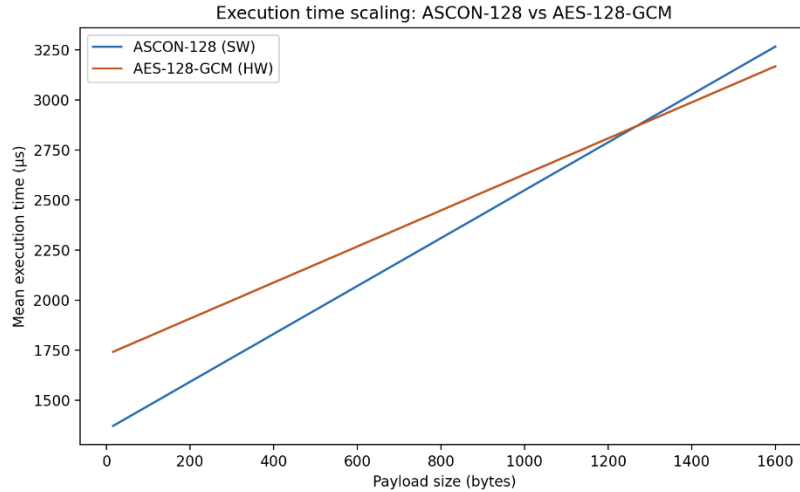


Figure 7- execution time scaling

4.3.2 Throughput

To evaluate whether execution time scaled linearly with payload size, a linear regression of mean execution time against payload size was performed for each algorithm. Both algorithms exhibited near-perfect linear scaling: ASCON-128 produced a slope of 1.197 $\mu\text{s}/\text{byte}$ with $R^2 = 0.9999997$ (essentially 1.0), and AES-128-GCM produced a slope of 0.901 $\mu\text{s}/\text{byte}$ with $R^2 = 0.9999998$. The lower slope for AES-128-GCM indicated that, once its larger fixed per-operation cost was accounted for, the hardware-accelerated algorithm added less additional time per byte of payload than ASCON-128's software implementation. This is the numerical basis for the execution time crossover visible in Figure 7, where AES-128-GCM's higher fixed cost was progressively offset by its lower scaling for ASCON-128 is discussed further in relation

to the sponge-duplexing scaling behavior described in 2.2.5. These times include instrumentation (5.5.1); their increments per byte were confirmed in Campaign B (1.180 and 0.897 $\mu\text{s/B}$).

Algorithm	Time slope ($\mu\text{s}/\text{byte}$)	Time R^2	Energy slope ($\mu\text{J}/\text{byte}$)	Energy intercept (μJ)	Energy R^2
ASCON-128	1.197	0.99999997	0.1068	15.03	0.9105
AES-128-GCM	0.901	0.99999998	0.0335	30.14	0.4275

Table 4-Time and energy regression results

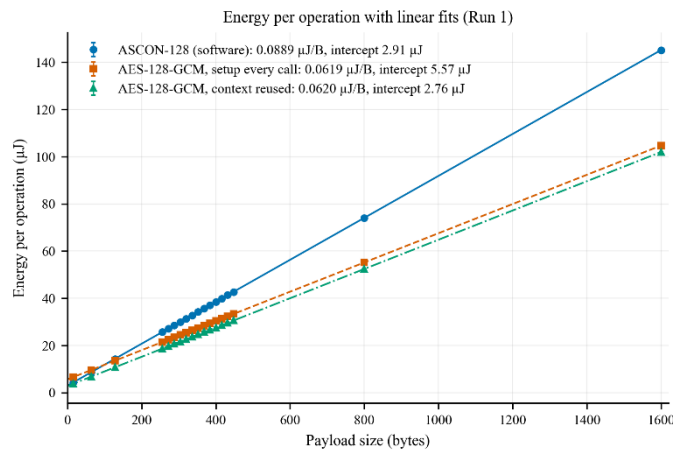


Figure 8-Regression scaling overlay

4.4 Power Consumption Results

Mean corrected power (4.2.2) was computed per payload size for both algorithms. At the smallest payload size (16 bytes), ASCON-128 drew a mean power delta of 20.3mW while AES-128-GCM drew 24.2mW. At 800 bytes, this gap widened considerably: ASCON-128 drew 52.2mW while AES-128-GCM drew only 22.2mW – a reversal from the smallest payload size, at which AES-128-GCM had drawn more power. At the largest payload size (1,600 bytes), ASCON-128 drew 52.8mW against AES-128-GCM’s 48.1mW.

This pattern indicated that ASCON-128’s power drew scaled upward more steeply with payload size than AES-128-GCM’s, consistent with ASCON-128’s operations remaining entirely CPU-bound throughout the sponge permutation, while AES-128-GCM’s hardware engine absorbed a larger share of the computational load into dedicated silicon rather than CPU as payload size increased. The consequence of this

power scaling difference for per-byte energy efficiency, which is the metric most directly relevant to objectives 1 and 2, is presented in 4.5.

The power figures of this section are not used for conclusions: comparison with Campaign B (Table 16) showed that they were 20 to 25 mW at small payload sizes against 64 to 75 mW measured with the sustained-load method, and that they contained steps that do not exist in the validated data (5.5.1).

4.5 Per-Byte Energy Efficiency and Break-Even Analysis

4.5.1 Per-Byte Energy Curves

Per-operation energy was computed as the product of mean power (mW) and mean execution time (ms) for each payload size, and per-byte energy was computed by dividing this figure by the payload sizes in bytes. At the smallest payload size (16 bytes), ASCON-128 consumed 1.742 $\mu\text{J}/\text{byte}$ of active-phase energy, compared with AES-128-GCM's 2.633 $\mu\text{J}/\text{byte}$. ASCON-128 was more efficient at this payload size. This gap narrowed rapidly as payload size increased: by 336 bytes, ASCON-128 consumed 0.107 $\mu\text{J}/\text{byte}$ against AES-128-GCM's 0.150 $\mu\text{J}/\text{byte}$, and by 352 bytes the ordering has reversed, with ASCON-128 at 0.106 $\mu\text{J}/\text{byte}$ against AES-128-GCM's now -lower 0.086 $\mu\text{J}/\text{byte}$. At the largest payload size tested (1,600 bytes), ASCON-128 consumed 0.108 $\mu\text{J}/\text{byte}$ compared with AES-128-GCM's 0.095 $\mu\text{J}/\text{byte}$.

Adding this sleep-phase energy constant from 4.2.4 (990 μJ , identical for both algorithms at every payload size) produced the total duty-cycle energy per byte; the figure most directly relevant to Objectives 1 and 2, since it captured the complete Sleep $\rightarrow$ Wake $\rightarrow$ Sense $\rightarrow$ Encrypt $\rightarrow$ Transmit $\rightarrow$ Sleep cycle rather than the active encryption phase alone. Because the sleep-phase term was constant across payload size, it shifted the absolute level of both curve upward without altering their shape or the location of the break-even point identified in 4.5.2 at 16 bytes, ASCON-128's total duty cycle energy was 63.62 $\mu\text{J}/\text{byte}$ against AES-128-GCM's 64.51 $\mu\text{J}/\text{byte}$; at 1,600 bytes, ASCON-128's total figure had fallen to 0.727 $\mu\text{J}/\text{byte}$ against AES-128-GCM's 0.714 $\mu\text{J}/\text{byte}$.

The per-byte energies and sleep-inclusive figures of this section are superseded by the validated values in 5.4.5 and 5.4.8. The sleep term (990 μJ) is a modelled constant and identical for both ciphers.

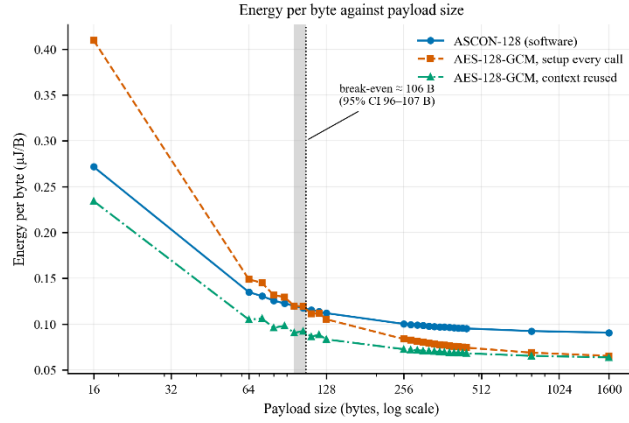


Figure 9- Energy per byte comparison

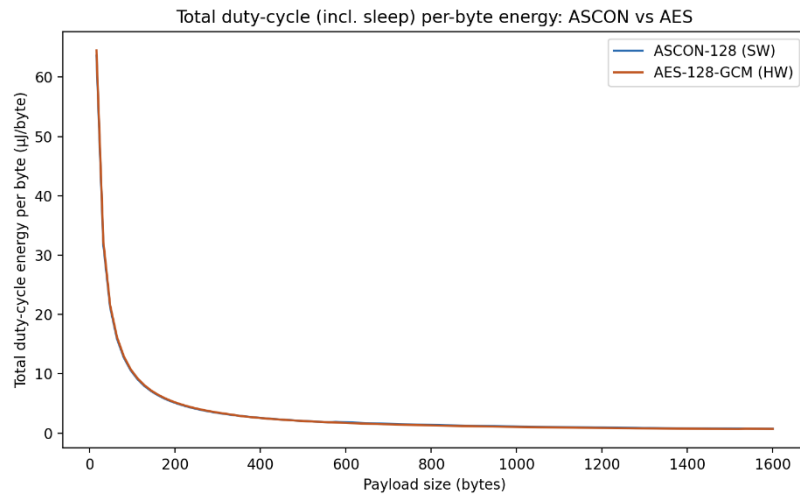


Figure 10-Total duty cycle energy per byte

4.5.2 Break-Even / Crossover Payload Size

The break-even point size was identified by computing, for each of the 100 payload sizes, the difference

$$\text{'ascon_minus_aes_energy'} = \text{energy_per_byte_uJ(ASCON)} - \text{energy_per_byte_uJ(AES)}$$

and locating the sign change in this difference across ascending payload size. A clean sign change was identified between the 336-byte payload size (difference = - 0.043 $\mu\text{J}/\text{byte}$, ASCON-128 more efficient) and the 352-byte payload size (difference = +0.020 $\mu\text{J}/\text{byte}$, AES-128-GCM more efficient). Linear interpolation between these

two bracketing sizes gave an interpolated break-even payload size of approximately 347 bytes. This crossover point was identical whether computed from active-phase energy or from total duty-cycle energy (including the sleep-phase term), since the sleep-phase term was a constant added identically to both curves and therefore could not move a sign change present in the active-phase difference.

A second much smaller sign change was detected between 528 and 560 bytes, where the difference between the two curves oscillated within a band of approximately ± 0.003 $\mu\text{J}/\text{byte}$; less than 3% of the magnitude of the 336–352-byte crossing. This was interpreted as the two curves converging to near-parity in that region rather than as a second, genuine efficiency-regime change, and is reported here as such rather than as an additional break-even point.

This 347-byte figure was reassessed in 5.5.2: it coincides with a step in the measured AES power between 336 and 352 bytes and was not confirmed by the sustained-load measurement.

Payload size (bytes)	ASCON-128 ($\mu\text{J}/\text{byte}$)	AES-128-GCM ($\mu\text{J}/\text{byte}$)	ASCON – AES ($\mu\text{J}/\text{byte}$)
320	0.112	0.162	-0.050
336	0.107	0.150	-0.043
352	0.106	0.086	+0.020
368	0.103	0.085	+0.018

Table 5-Comparison by size

4.5.3 Regime Segmentation

To relate the break-even results directly to realistic payload regimes, the comparison dataset was segmented into a small-payload regime (< 256 bytes, representative of narrow-band LPWAN-style traffic) and a large-payload regime (> 800 bytes, representative of bulk-telemetry traffic). In the small-payload regime, ASCON-128 was more efficient on average, with a mean difference of -0.225 $\mu\text{J}/\text{byte}$ (ASCON-128 lower). In the large-payload regime, AES-128-GCM was more efficient on average, with a mean difference of $+0.065$ $\mu\text{J}/\text{byte}$ (AES-128-GCM lower). This provided the direct, data-based answer to Objective 3's requirement to identify the conditions under which software-based ASCON-128 remained preferable to hardware-accelerated AES-128-GCM.

Regime	Mean (ASCON – AES) energy/byte	More energy-efficient algorithm
--------	--------------------------------	---------------------------------

Small (< 256 B, LPWAN-typical)	-0.225 $\mu\text{J}/\text{byte}$	ASCON-128
Large (> 800 B, bulk-telemetry-typical)	+0.065 $\mu\text{J}/\text{byte}$	AES-128-GCM

Table 6-Regime segmentation

4.6 Memory Footprint Results

Total memory footprint was calculated per trial as the sum of peak heap usage and stack usage, and averaged across all payload sizes for each algorithm. ASCON-128 recorded a mean memory footprint of 2,772.8 bytes, while AES-128-GCM recorded a mean memory footprint of 3,441.0 bytes; approximately 668 bytes, or 24%, higher than ASCON-128. This difference was broadly constant across payload sizes rather than scaling with payload size (2,771.0 bytes at 16 bytes payload versus 2,767.6 bytes at 1,600 bytes payload for ASCON-128; 3,463.2 bytes versus 3,449.5 bytes for AES-128-GCM over the same range), consistent with the memory-footprint methodology described in 3.5.4 and supporting its treatment as a discrete, largely size-independent comparison rather than a scaling relationship. This is consistent with AES-128-GCM's use of the mbedTLS library's cryptographic context structures, which carry a fixed memory overhead independent of the amount of data being encrypted. This measurement does not depend on the timing and power method examined in Chapter 5.

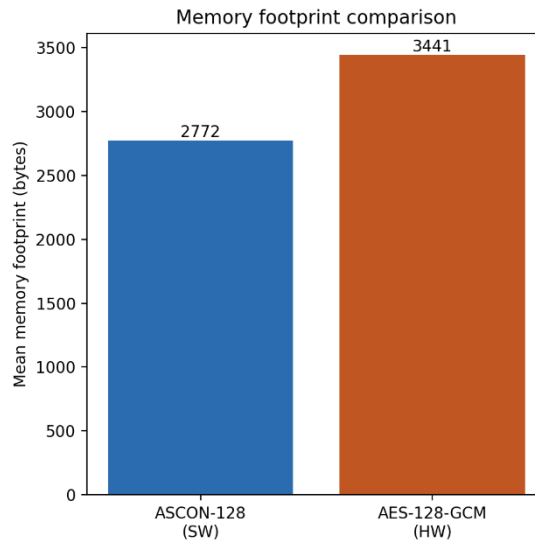


Figure 11- Memory footprint comparison

4.7 Statistical Significance Testing

4.7.1 Per-Size Hypothesis Testing

The tests of this section were one-tailed and uncorrected. They are re-run two sided and with Holm correction in 5.5.6 (Table 17). Following 3.7.2 one-tailed independent (Welch's) t-tests were conducted at a significance level of $\alpha = 0.01$, run separately for each of the 100 payload sizes, for execution time, power and energy per byte. Rather than reporting a single pooled significance figure, the proportion of payload sizes at which the difference between algorithms was statistically significant was reported for each metric, consistent with the 100-payload-level scaling design described in 3.3.2.

4.7.2 Regression Summary

The linear regression of per-operation energy against payload size produced intercepts of 15.03 μJ for ASCON-128 and 30.14 μJ for AES-128-GCM (Table 4). An intercept is the value of the fitted line at a hypothetical zero-byte payload. It includes every fixed cost that does not depend on payload size, and with R^2 of 0.91 and 0.43 it is imprecise. It was therefore not taken as evidence of the AES hardware initialization cost. That cost was measured directly in Campaign B (1.90 μJ ; 5.4.3), and the intercept is called a fitted fixed cost (5.5.4).

4.8 Summary of Chapter 4

This chapter reported the results of a controlled, energy-centric comparison between software-based ASCON-128 and hardware-accelerated AES-128-GCM across a full IoT duty cycle, following the identical cleaning, calibration, and statistical pipeline described in 4.2 for both algorithms.

The significance tests of 4.7.1 were on-tailed and uncorrected and have been re-run two-sided with correction for multiple comparisons (5.5.6). the interpretation given in an earlier version of the section, that the efficiency gap was driven primarily by power draw rather than speed, is withdrawn. Chapter 5 audits the measurement method used for these results, reports a validation measurement, and states which findings are retained, changed or withdrawn.

Chapter 05: Validation of the Measurements

5.1 Introduction

The results reported in Chapter 4 came from a per-trial benchmark that is called Campaign A in this chapter. During the review of that work, five questions were raised about the analysis: the inconsistent flowchart symbols, the disagreement between the regression-based break-even (206 bytes) and the measured one (347 bytes), the interpretation of a regression intercept as hardware initialization energy, the removal of outliers, and the use of separate one-tailed t-tests without correction for multiple comparisons. Answering the second and third questions required examining how Campaign A actually measured time and power. That examination showed that the measurement method could not support energy conclusions for an operation lasting a few milliseconds, and a second campaign was therefore designed and carried out.

This chapter has four aims. It documents the audit of Campaign A (5.2). It describes and reports the second campaign, Campaign B, a sustained-load measurement designed to avoid the identified limitations (5.3 and 5.4). It re-examines the Campaign A results in the light of Campaign B and answers the five review questions (5.5). It then states how far the validation can be trusted (5.6) and which findings of Chapter 4 stand, change or are withdrawn (5.7). The interpretation of the corrected findings follows in Chapter 6.

5.2 Why Validation Was Needed: Audit of Campaign (A)

5.2.1 Method

The Campaign A firmware (`main.c`, `run_trial.c`, `encrypt_measure.c`, `power_monitor.c` and `ina219.c`) and its build configuration were inspected line by line, and the raw data were compared with what the code implies. Campaign B was then run and compared with Campaign A at the payload sizes that both campaigns share.

5.2.2 Findings

#	Property of Campaign A	Consequence
---	------------------------	-------------

1	Power was estimated as the difference of two instantaneous INA219 readings, one taken just before and one just after the cipher call.	The INA219 was configured for 12-bit conversions (register value 0x399F), which take about 532 μ s, and its registers hold the result of the most recently completed conversion. A call lasting about 2 ms is therefore not bracketed by the two readings in any defined way.
2	The "before" reading followed an MQTT publish that waited for a broker acknowledgement, so the radio was active.	The baseline used for the difference was not an idle baseline.
3	The timestamp pair enclosed the heap counters, two INA219 register reads, the creation of a FreeRTOS task for the cipher, a semaphore wait, and the cipher call itself.	The recorded time is trial latency including instrumentation, not the cipher time (5.5.1).
4	The AES-128-GCM path initialised the context and loaded the key inside every timed call. ASCON-128 has no equivalent step.	The measured time and energy of AES include a per-call setup that is not present for ASCON, which is realistic for a node that loses RAM in deep sleep, but the two are only comparable if this is stated.
5	Each trial ran once after a fresh boot, followed by a Wi-Fi reconnect.	Cold-start effects (empty instruction cache, first use of peripherals) are included in the trial, and are larger for the AES path than for ASCON (5.5.1).
6	ASCON-128 and AES-128-GCM were collected in separate sessions.	Session conditions (temperature, supply, Wi-Fi environment) can differ between the two datasets.
7	The firmware ran at 160 MHz.	Chapter 3 states 240 MHz; this is corrected.

Table 7- Audit of the Campaign A measurement method

5.3 Validation Design (Campaign B)

5.3.1 Principle

Campaign B measures the energy of the cipher call without the problems of Table 8. Instead of bracketing one call, it repeats the call back-to-back for 700 ms while the INA219 averages current, and compares that with the idle level measured immediately before and after. The extra current, multiplied by the bus voltage and the time per call, gives the energy per call. The INA219 was configured for 128-sample averaging on the shunt channel (about 68 ms per result, register value 0x39FF), so a 700 ms window contains several complete, clean results. Because every reading contains the same constant sensor offset, the offset cancels in the load-minus-idle difference.

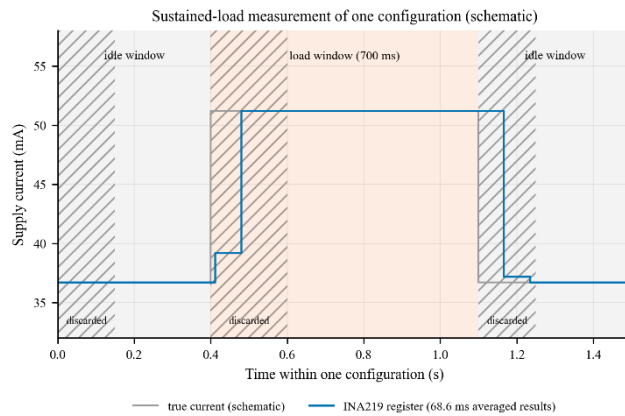


Figure 12 - Sustained load measurement of one configuration

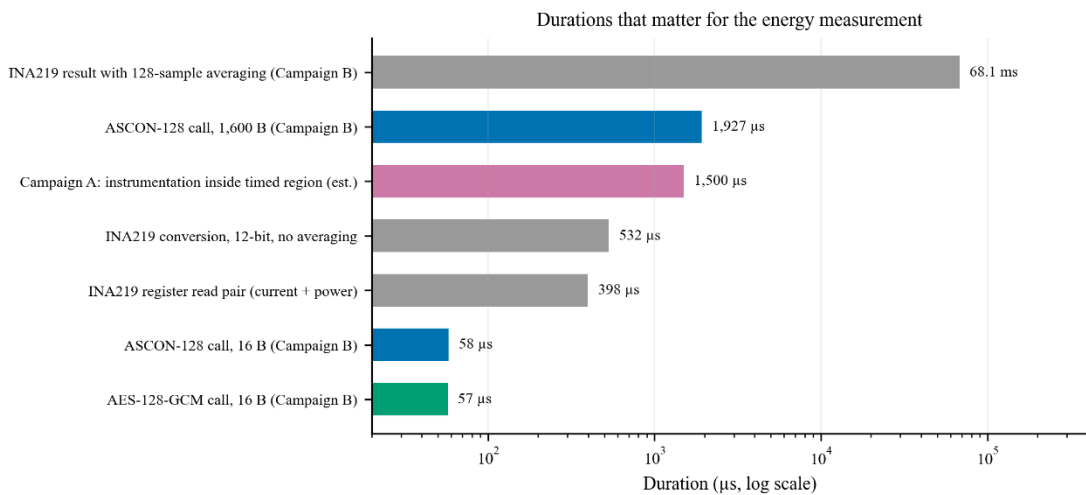


Figure 13 - Sensor timing scale

5.3.2 Procedure

The firmware is a separate program (``validation_fw``) that reuses the ASCON-128 source file of Campaign A and the same key, payload generator and build settings. It does not use Wi-Fi, MQTT or deep sleep. For each configuration (a cipher variant and a payload size):

- Idle window, 400 ms, with the worker task blocked. The first 150 ms are discarded so that the INA219 averaging window has settled. Mean current $I_{idle,pre}$ is taken from the remainder.
- Load window, 700 ms. A worker task pinned to core 1 calls the cipher back-to-back and counts the calls and the elapsed time (``esp_timer_get_time()``). Core 0 polls the INA219 about every 10 ms. The first 200 ms (about three INA219 result periods) are discarded. Mean current I_{load} is taken from the remainder.
- Idle window, as in step 1, giving $I_{idle,post}$.

The energy per call is,

$$E_{op} = V_{bus} \times (I_{load} - \bar{I}_{idle}) \times t_{op}$$

Equation 4 - Energy per call equation

where $\bar{I}_{idle}$ is the mean of the two idle values, V_{bus} the mean measured bus voltage (about 4.72 V) and t_{op} the elapsed time divided by the number of calls. Per-byte energy is E_{op} divided by the payload size.

5.3.3 Variants

Variant	What one call does	Purpose
ASCON	<code>`ascon128_encrypt`</code> (software)	Lightweight cipher
AES_COLD	initialize GCM context, load key, encrypt and authenticate, release	The sequence used in Campaign A; the situation of a node that loses RAM in deep sleep
AES_WARM	encrypt and authenticate only; the context is prepared once before the window	AES with the setup amortized
AES_SETKEY	initialize context, load key, release, with no encryption	The setup cost in isolation

Table 8-Cipher variants in Campaign B

All configurations are run within each replicate in a shuffled order (fixed seed), so that the two ciphers share the same session, temperature and supply conditions.

5.3.4 Runs

Run 1: covered 18 payload sizes (16, 64, 128, 256 to 448 in steps of 16, 800 and 1,600 bytes) with 10 replicates (550 rows).

Run 2: was a fine sweep of 64 to 128 bytes in steps of 8, with 15 replicates (420 rows), designed after Run 1 located the crossing between 64 and 128 bytes. The setup-only variant was included in both runs.

5.3.5 Statistics

Per-configuration means, standard deviations and 95% confidence intervals (Student's t) were computed over replicates. Differences between ciphers were tested per payload size with a two-sided paired t-test on the per-replicate differences (each replicate contains both ciphers), with Holm correction across the sizes of a run (family-wise $\alpha = 0.01$). The break-even was found by linear interpolation of the sign change in the mean difference, and its 95% interval by a bootstrap of replicates (5,000 resamples, replicates kept paired across sizes). No outlier removal, filtering or offset correction was applied.

5.4 Validation Results

5.4.1 Data quality

All 970 rows (550 in Run 1, 420 in Run 2) were retained. The extra current under load was between 12.8 and 16.7 mA in every row. The two idle windows of a replicate differed by +0.006 mA on average in Run 1 and +0.009 mA in Run 2 (standard deviations 0.07 and 0.11 mA). Across replicates the median coefficient of variation was 0.3% for energy per call and below 0.001% for time per call.

5.4.2 Time per operation and throughput

Payload (B)	ASCON (μ s)	AES_COLD (μ s)	AES_WARM (μ s)
16	57.9	102.8	57.4
64	114.5	145.8	100.5
128	190.0	203.2	157.9
352	454.3	404.1	358.9
800	982.8	806.0	760.8

1600	1926.6	1523.5	1478.5
------	--------	--------	--------

Table 9- Time per operation

Time was linear in payload size for all variants Table 9 ($R^2 > 0.9999$): ASCON-128 added 1.180 μs per byte (intercept 39.0 μs), AES-128-GCM 0.897 μs per byte (intercepts 88.4 μs with setup on each call, 43.1 μs with the context reused). Throughput at 1,600 bytes was 0.83 MB/s for ASCON-128 and 1.05 MB/s (setup each call) or 1.08 MB/s (context reused) for AES-128-GCM. ASCON-128 was faster below about 175 bytes (interpolated between 128 and 256 bytes) and AES-128-GCM above it (Figure 14).

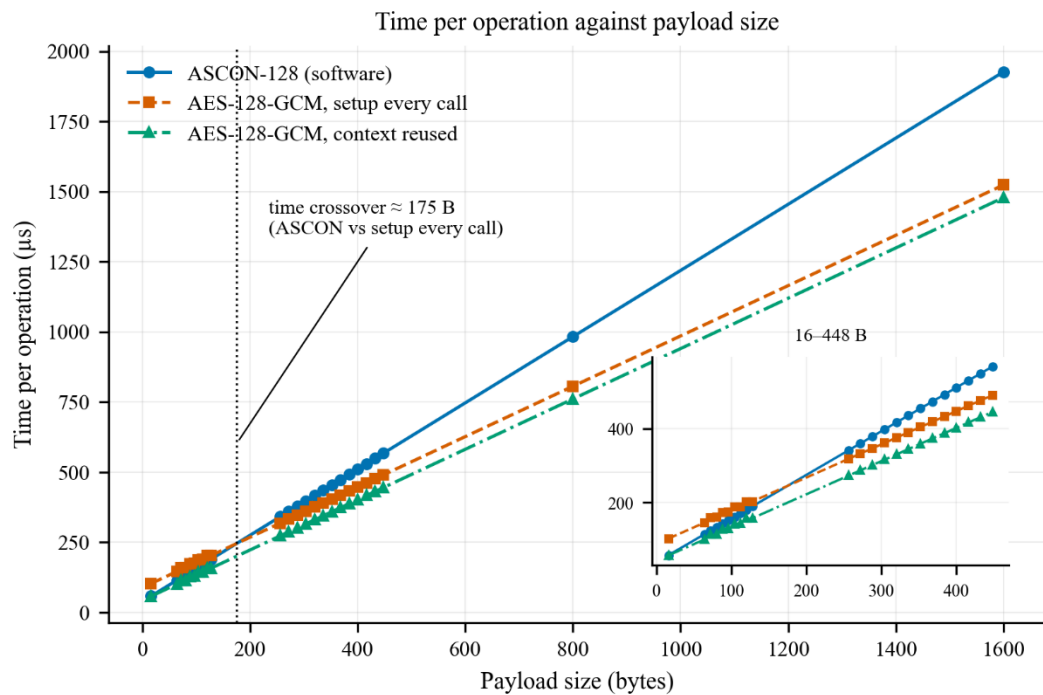


Figure 14 - Time per operation

5.4.3 Setup cost of the AES-GCM context

The setup step alone took 30.9 μs and 1.90 μJ per call (95% confidence interval for energy 1.89 to 1.90 μJ), and the value was the same in Run 1 and Run 2 to three digits. The difference between the AES_COLD and AES_WARM fitted intercepts was 45.4 μs and 2.82 μJ , so the

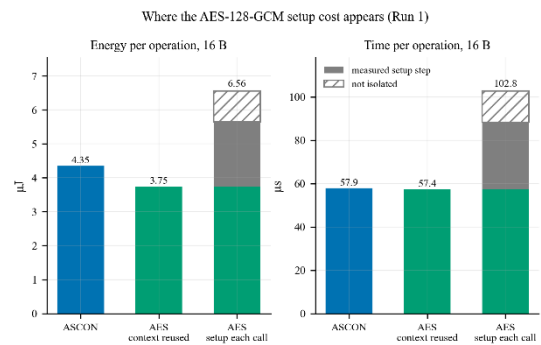


Figure 15 - AES setup cost

measured setup step accounts for about two thirds of it; the remainder was not isolated (Figure 15).

5.4.4 Extra power

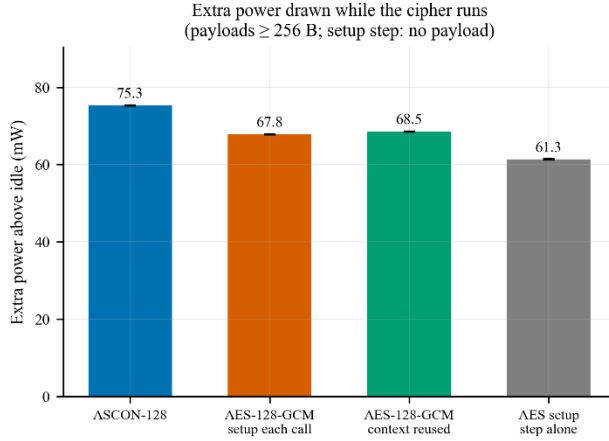


Figure 16 - Power consumption by variant

Averaged over payload sizes of 256 bytes and above, the extra power above idle was 75.3 mW for ASCON-128, 67.8 mW for AES-128-GCM (setup each call) and 68.5 mW for AES-128-GCM. The setup step drew 61.3 mW. ASCON-128 power was constant over payload size (75.1 to 75.4 mW), and AES power rose slightly with

size (63.8 mW at 16 bytes to 68.7 mW at 1,600 bytes) as the setup was amortized (Figure 16). Idle current with Wi-Fi off was about 35 to 37 mA at 4.72 V.

5.4.5 Energy per operation and per byte

Payload (B)	ASCON (μJ)	AES_COLD (μJ)	AES_WARM (μJ)	ASCON (μJ/B)	AES_COLD (μJ/B)	AES_WARM (μJ/B)
16	4.35	6.56	3.75	0.272	0.410	0.234
64	8.62	9.54	6.73	0.135	0.149	0.105
128	14.30	13.48	10.69	0.112	0.105	0.084
352	34.19	27.36	24.59	0.097	0.078	0.070
800	74.01	55.13	52.36	0.093	0.069	0.065
1600	145.17	104.67	101.94	0.091	0.065	0.064

Table 10- Energy per operation and per byte

Variant	Energy Slope (μJ/B)	Energy intercept (μJ)	R ²
ASCON	0.0889	2.91	> 0.9999
AES_COLD	0.0619	5.57	> 0.9999
AES_WARM	0.0620	2.76	> 0.9999

Table 11- Linear fits of energy per call against payload size

Table 10 reports energy per operation and per byte for each variant at six representative payload sizes, from 16 to 1,600 bytes. At 16 bytes ASCON-128 used 34% less energy than AES-128-GCM with setup on each call; at 1,600 bytes AES-128-GCM used 28% less. The three fitted lines are shown in Figure 17.

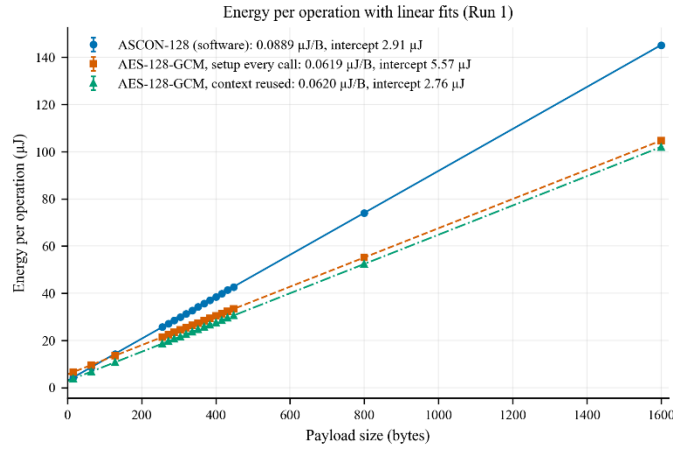


Figure 17 - Energy per operation with linear fits (Run 1)

5.4.6 Break-even payload size

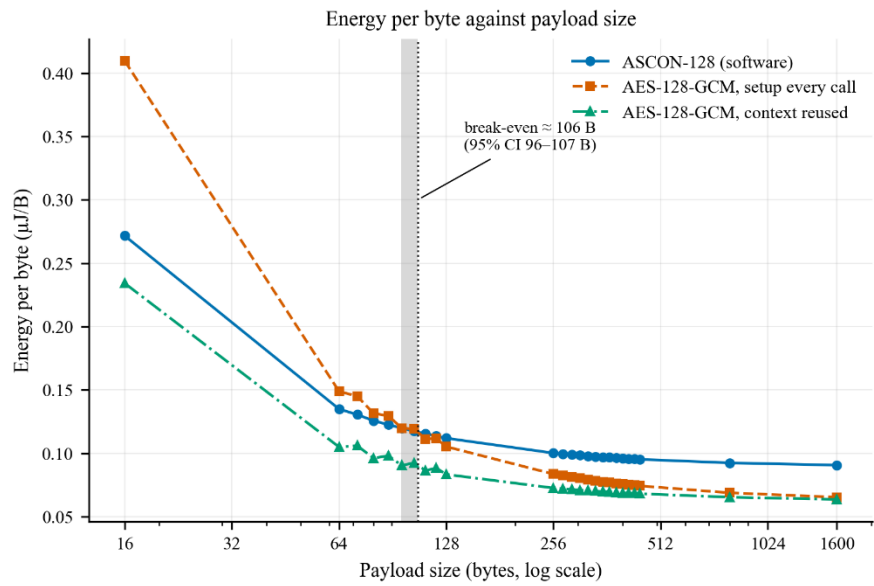
The difference $\text{ASCON} - \text{AES_COLD}$ was $-0.93 \mu\text{J}$ at 64 bytes and $+0.81 \mu\text{J}$ at 128 bytes in Run 1. The fine sweep of Run 2 located the crossing:

Payload (B)	ASCON – AES_COLD (μJ)	Result (Holm-corrected, $\alpha = 0.01$)
64	- 0.871	ASCON lower
72	-1.044	ASCON lower
80	-0.465	ASCON lower
88	-0.617	ASCON lower
96	-0.007	not significant ($p = 0.78$)
104	-0.196	ASCON lower
112	+0.467	AES lower
120	+0.234	AES lower
128	+0.877	AES lower

Table 12-Fine sweep around the break-even

Interpolation gave a break-even of 106 bytes, with a 95% bootstrap interval of 96 to 107 bytes; a clean crossing occurred in every resample (Figure 18 and Figure 19). At 96 bytes the ciphers could not be distinguished. Sizes that are not multiples of 16 (72, 88, 104, 120) fall slightly below their neighbors for AES, consistent with 16-byte

blocks; this explains the small zig-zag and the asymmetry of the interval. Where the two runs overlapped (64 and 128 bytes), mean ASCON energy differed by 0.04 and 0.07 μJ



(below 0.5%) and AES_COLD by 0.01 and 0.002 μJ .

Figure 18 - Energy per byte against payload size

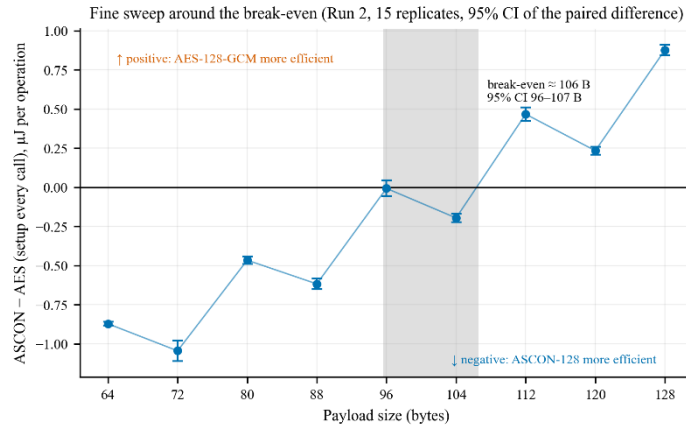


Figure 19 - Fine sweep, run 2

The fitted lines of Table 11 give an independent estimate: $(5.57 - 2.91) / (0.0889 - 0.0619) \approx 99$ bytes, in agreement with the direct measurement. The break-even is therefore approximately 100 bytes (96 to 107 bytes) when AES-128-GCM sets up its context on each message.

5.4.7 Effect of reusing the AES context

With the context reused (AES_WARM), AES-128-GCM used less energy than ASCON-128 at every size from 64 bytes upward (by 1.9 μJ at 64 bytes and 43.2 μJ at 1,600 bytes). At 16 bytes the two were within 0.6 μJ , with times of 57.4 and 57.9 μs . No break-even exists in that case.

5.4.8 Encryption within a modelled duty cycle

The sleep phase was modelled as $E_{\text{sleep}} = 3.3 \text{ V} \times I_{\text{sleep}} \times 60 \text{ s}$ (Equation 3), with $I_{\text{sleep}} = 5 \mu\text{A}$ (990 μJ) and $10 \mu\text{A}$ (1,980 μJ) shown as two cases.

Payload (B)	ASCON active (μJ)	AES_COLD active (μJ)	Difference, % of cycle (5 μA)	Difference, % of cycle (10 μA)
16	4.35	6.56	0.22% (AES higher)	0.11%
352	34.19	27.36	0.67% (ASCON higher)	0.34%
1600	145.17	104.67	3.57% (ASCON higher)	1.9%

Table 13 - Effect of the cipher on modelled cycle energy

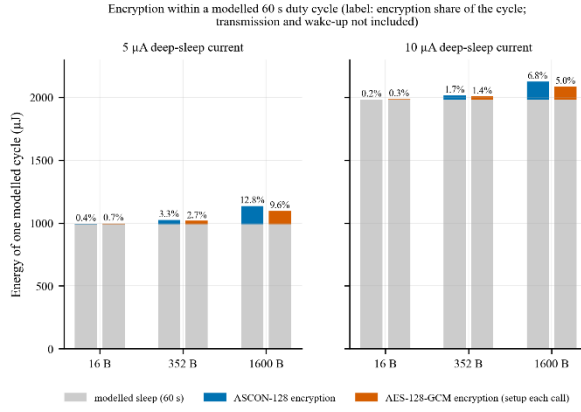


Figure 20 - Encryption share of the modelled cycle at each payload cycle

Figure 20 shows the encryption share of the modelled cycle at each payload size. Encryption was 0.4% of the modelled cycle at 16 bytes and 13% at 1,600 bytes (ASCON-128, 5 µA case), and the cipher choice changed total cycle energy by under 4% at all tested sizes.

5.5 Re-examination of the Campaign A Results

5.5.1 Side-by-side comparison

Campaign A and Campaign B share eight payload sizes. Time in Campaign A exceeded Campaign B by a nearly constant amount for each cipher:

Payload (B)	ASCON A (µs)	ASCON B (µs)	Difference	AES A (µs)	AES_COLD B (µs)	Difference
16	1370.9	57.9	1313	1740.8	102.8	1638
64	1428.3	114.5	1314	1784.5	145.8	1639
128	1504.6	190.0	1315	1841.6	203.2	1639
352	1772.7	454.3	1318	2043.9	404.1	1640
448	1887.7	567.6	1320	2130.4	490.2	1640
1600	3266.5	1926.6	1340	3167.6	1523.5	1644

Table 14 - Time per operation in Campaign A and Campaign B

Three observations follow. First, the slopes agree: 1.197 against 1.180 µs/B for ASCON-128 and 0.901 against 0.897 µs/B for AES-128-GCM (differences of 1.4% and 0.4%). The time measurements of Campaign A are therefore sound as increments. Second, the offset is about 1.31 ms for ASCON-128 and 1.64 ms for AES-128-GCM, which is the instrumentation and cold-start cost of Table 8. Third, the offset is about 325 µs larger for AES-128-GCM, which is far more than the measured setup step (30.9 µs). The most plausible contributor is a first-execution effect after boot, which was not tested and is proposed as future work (7.5). The time crossover of Campaign A near

1,250 bytes is largely produced by these 325 μ s differences in offsets (it would lie near 1,150 bytes from the offset difference and the slope difference alone), and the crossover of the cipher calls themselves is near 175 bytes (5.4.2).

Power and energy did not agree Table 15:

Payload(B)	ASCON A Power (mW)	ASCON B power (mW)	AES A power (mW)	AES_COLD B power (mW)	ASCON A energy (μ J)	ASCON B energy (μ J)	AES A energy (μ J)	AES_COLD B energy (μ J)
16	20.3	75.1	24.2	63.8	27.9	4.35	42.1	6.56
64	19.8	75.2	26.0	65.4	28.3	8.62	46.3	9.54
128	19.7	75.2	25.1	66.4	29.7	14.30	46.3	13.48
256	19.8	75.3	25.7	67.4	32.8	25.66	50.4	21.42
352	21.1	75.3	14.8	67.7	37.3	34.19	30.2	27.36
448	20.4	75.2	16.8	68.0	38.6	42.70	35.9	33.32
800	52.2	75.3	22.2	68.4	120.5	74.01	54.3	55.13
1,600	52.8	75.4	48.1	68.7	172.6	145.17	152.4	104.67

Table 15 - Extra power and energy per operation in Campaign A and Campaign B

Campaign A power was 20 to 25 mW for both ciphers at small sizes against 64 to 75 mW in Campaign B, and it changed in steps that Campaign B does not show: ASCON-128 from 19.8 to 52.6 mW between 560 and 576 bytes, AES-128-GCM from 24.9 to 14.8 mW between 336 and 352 bytes, and from 21.9 to 47.8 mW between 1,520 and 1,536 bytes (Figure 21). A constant per-call power such as the one measured in Campaign B does not change in steps with payload size. Consequently, Campaign A energy at 16 bytes was 6.4 times the validated value for both ciphers, and at 1,600 bytes 19% (ASCON) and 46% (AES) too high.

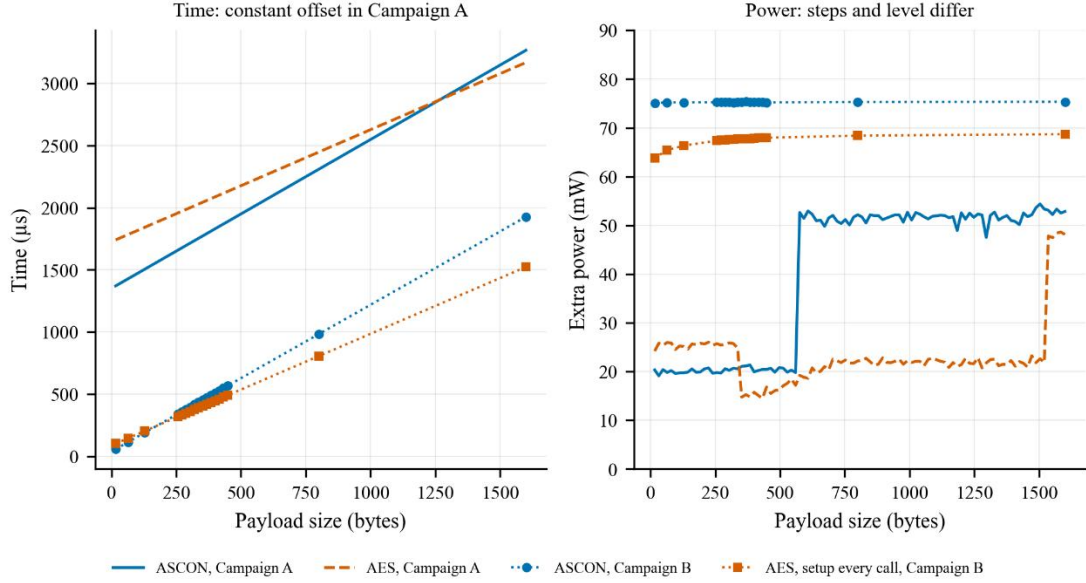


Figure 21 - Campaign A's stepped power/energy vs Campaign B's flat

5.5.2 The origin of the 347-byte break-even

The Campaign A energy per byte of AES-128-GCM fell abruptly from 0.150 $\mu\text{J/B}$ at 336 bytes to 0.086 $\mu\text{J/B}$ at 352 bytes (a 43% drop), whereas AES time rose smoothly from 2,029 to 2,044 μs over the same step. The whole drop is caused by the measured AES power falling from 24.85 to 14.75 mW (−10.1 mW, −41%). ASCON-128 energy per byte was 0.107 $\mu\text{J/B}$ at 336 bytes and 0.106 $\mu\text{J/B}$ at 352 bytes. The sign change of the ASCON – AES difference, and hence the 347-byte break-even, therefore coincides exactly with this discontinuity.

5.5.3 Regression versus measured break-even

The Campaign A regression lines (Table 4) give a break-even of $(30.14 - 15.03) / (0.1068 - 0.0335) \approx 206$ bytes. A straight line was fitted to data that contained steps (AES at 340 bytes, ASCON at 570 bytes), which is why the energy fits had R^2 of only 0.43 for AES and 0.91 for ASCON. A line fitted through a stepped curve does not reproduce the crossing of the curve. The earlier statement that the difference was "expected" because of noise was therefore incomplete; the disagreement was a symptom of the measurement problem. In Campaign B the regressions have $R^2 > 0.9999$ and the fitted (99 bytes) and directly measured (96 to 107 bytes) break-evens agree.

5.5.4 The meaning of the regression intercept

A regression intercept is the value the fitted line takes at a payload of zero bytes, below the smallest size measured (16 bytes). It absorbs every fixed cost that does not depend on payload size, including measurement overhead, and it inherits the poor fit of the line ($R^2 = 0.43$ for AES in Campaign A). The Campaign A intercept of 30.14 μJ could not be attributed to hardware initialization. Campaign B measured the initialization step directly at 1.90 μJ (5.4.3), about one sixteenth of that figure. What survives from Campaign A is the ratio of the fitted fixed costs (30.14 to 15.03, about 2.0), which in Campaign B is 5.57 to 2.91 μJ (1.9). In this dissertation an intercept is therefore called a fitted fixed cost, and only the measured setup step is called the initialization cost.

5.5.5 Outlier trimming

Campaign A trimmed 18.5% of the ASCON-128 rows and 15.9% of the AES-128-GCM rows (IQR rule with $k = 1.5$, per payload size, on time and power). The analysis was repeated without trimming and with $k = 3$ (calibration constants held fixed) Table 16:

	No Trimming	IQR $k=3.0$	IQR $k = 1.5$ (used)
Rows (ASCON / AES)	56,100 / 51,000	53,264 / 47,301	45,735 / 42,868
Energy break-even (B)	345.0	347.4	346.8
Time crossover (B)	1,245	1,267	1,267
Sign changes of energy difference	8	4	3

Table 16 - Sensitivity of campaign A to outlier trimming

Calibration constants were held fixed across all three variants for comparability; the $k = 1.5$ row counts therefore differ from Table 3 by up to 10 rows. The break-even was 345 to 347 bytes in all three cases. Trimming therefore did not create the result, and it is not the explanation for the difference with Campaign B: the step of 5.5.2 is a systematic effect present in the untrimmed data as well (Figure 22). Trimming did affect the noisiness of the curve (8 sign changes without trimming, 3 with). Campaign B

involved no trimming, because it involves no calibration constant and no data-cleaning step that needed one.

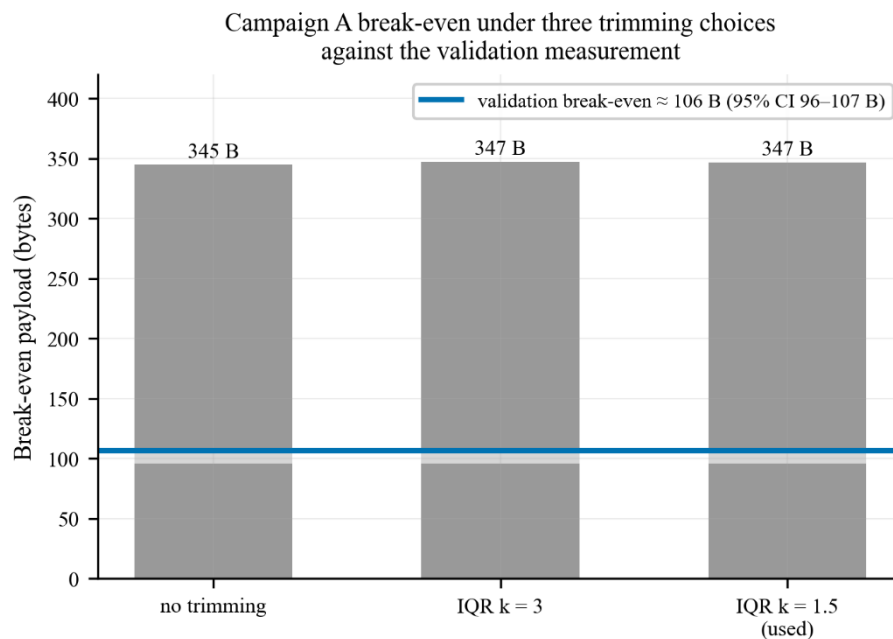


Figure 22 - Break even under three trimming choices (none, $k=3$, $k=1.5$)

5.5.6 Statistical testing

Campaign A applied 300 separate one-tailed Welch t-tests (100 sizes, three metrics) at $\alpha = 0.01$ without adjustment for multiple comparisons, with the alternative hypothesis fixed as "ASCON greater than AES" at every size. Both choices were inappropriate: with 300 tests about three false positives are expected by chance alone, and the direction of the difference reverses across the break-even, so a single one-sided hypothesis holds only on one side of it. The tests were repeated two-sided and Holm-corrected Table 17:

Sizes significant of 100 ($\alpha = 0.01$)	Time	Power	Energy per byte
One-tailed, as originally reported	21	78	72
Two-sided, uncorrected	100	99	93
Two-sided, Holm- corrected	100	99	92

Table 17 - Campaign A per-size tests

The originally reported "execution time significant at only 21 sizes" reflected the fixed direction: ASCON-128 is slower than AES-128-GCM only above the time crossover near 1,250 bytes, which covers about 21 of the 100 sizes. Two-sided, the times differed at every size. The inference in Chapter 4 that the two execution times "converged" and that the energy difference was driven by power and not speed is therefore withdrawn. In Campaign B, the tests were two-sided, paired and Holm-corrected, and effect sizes are reported next to them.

5.6 Reliability and Validity of the Validation

- Signal and noise: The extra current under load (12.8 to 16.7 mA) exceeded the drift of the idle baseline (about 0.01 mA on average, standard deviation about 0.1 mA) by two orders of magnitude.
- Repeatability: The median coefficient of variation of energy across replicates was 0.3%. Two separate runs with different sizes and replicate counts agreed to within 0.5% at the overlapping sizes, and the AES setup cost was 1.896 and 1.895 μJ .
- Internal consistency: Time and energy were linear in payload size ($R^2 > 0.9999$), and the break-even predicted by the fitted lines (99 bytes) agreed with the measured one (96 to 107 bytes). Campaign A failed this test (206 against 347 bytes).
- Independent paths: Time comes from the CPU timer and current from the INA219. Energy ordering followed time ordering with a nearly constant power difference, and the increments of time agreed with Campaign A to within 1.4%.
- Design instead of clean-up: The sensor offset cancels in the difference, the timed region contains only the cipher loop, the ciphers share one session in shuffled order, and no data were removed.

What the validation does not show: No independent reference instrument was used, so the absolute energy scale depends on the INA219 calibration, although the comparison between ciphers does not (same sensor, same offset). One board was used. The measurement is steady-state with Wi-Fi off, on one core, and does not include cold-start effects such as those suggested by the 325 μs difference of 5.5.1. Only the encryption of a payload with a 16-byte tag and no associated data was measured. Which parts of

GCM run in the AES peripheral and which on the CPU was not established. Sleep, wake-up and transmission were not measured.

5.8 Summary of Chapter 5

Campaign A, the per-trial benchmark of Chapter 4, was audited and found unable to resolve the energy of a call lasting a few milliseconds: it used two instantaneous sensor readings, a timed region containing instrumentation, and separate sessions for the two ciphers. Campaign B, a sustained-load measurement with interleaved ciphers and no data removal, measured the AES-128-GCM setup step directly (30.9 μ s, 1.90 μ J), found a break-even of about 100 bytes (96 to 107 bytes) when the setup is paid on every message, and found no break-even when the context is reused. Comparison with Campaign A showed that its time increments were sound, that its time offsets and power values were not, and that its 347-byte break-even coincides with a step in the measured AES power. The five review questions were answered: the regression disagreement was a symptom of that step, the intercept was not an initialization energy, trimming did not create the result but the result was nonetheless not valid, and the statistical tests were redone two-sided and with correction for multiple comparisons. Chapter 6 interprets the corrected findings.

Chapter 06: Discussion

6.1 Introduction

This study asked whether software-based ASCON-128 or hardware-accelerated AES-128-GCM is the more energy efficient choice for a battery-powered IOT node on the ESP32, and under what conditions (1.7). the answer, after validation (Chapter 5), is conditional on two things: the payload size, and whether the AES-128-GCM context is set up on every message. When it is, as on a node that loses RAM in deep sleep and sends one message per wake, ASCON-128 used less energy below about 100 bytes and AES-128-GCM above it. When the context is kept, AES-128-GCM was at least as efficient at every tested size. This replaces the break-even of 347 bytes reported in Chapter 4. This chapter interprets the corrected result (6.2), explains why the first analysis differed (6.2.4), compares the results with the literature (6.3) and states the limitations (6.4).

6.2 Interpretation of Findings

6.2.1 Why a break-even point exists at all

The break-even exists because the two ciphers pay for their work in different proportions. Writing energy per byte as $(\text{fixed cost} \div \text{payload size}) + \text{variable cost per byte}$ (Equation 5), the fixed term shrinks as the payload grows: small payloads are decided by the fixed cost and large payloads by the variable cost. Campaign B gave fixed costs (fitted intercepts) of 2.91 μJ for ASCON-128 and 5.57 μJ for AES-128-GCM with setup on each call, and variable costs of 0.0889 and 0.0619 μJ per byte (Table 11). Because each win on a different term, a payload size exists at which they are equal; solving the lines gives 99 bytes, and the directly measured crossing was 106 bytes (95% interval 96 to 107 bytes), in agreement.

The slopes have a plausible cause: ASCON-128 executes its permutation on the CPU, which stays busy longer (1.18 μs per byte against 0.90 for AES-128-GCM), whereas the AES block operation is performed by the AES peripheral. Which parts of GCM run in the peripheral and which on the CPU was not examined, so the AES slope is the measured cost of the complete library call.

The directly measured setup step (context initialization and key load, no encryption) cost 30.9 μs and 1.90 μJ . This is the measured initialization cost of the AES-GCM context. It replaces the figure of 30.14 μJ of the earlier draft, which was extrapolated from a poor regression (5.5.4). The difference between the AES_COLD and AES_WARM intercepts (2.82 μJ) is larger than the setup step, and the remainder was not isolated. The fitted fixed cost of ASCON-128 is the intercept of a fit, not a separately measured step.

Setting the context up on each message corresponds to a node that loses RAM in deep sleep and rebuilds its security state on every wake [Gerndt et al., 2025]; keeping it corresponds to a node that sends several messages per wake or stores the key schedule in RTC memory. The choice moves the break-even from about 100 bytes to no break-even, and is the practical form of the result.

Item	Content
AES-128-GCM's fixed cost	Preparing the context: initialization and loading of the key. Measured: 30.9 μs , 1.90 μJ per call.
AES-128-GCM per-byte cost	Block processing by the AES peripheral and the surrounding library calls. Measured: 0.897 μs and 0.062 μJ per byte.
ASCON-128 fixed cost	Initial permutation over key and nonce on the CPU. Fitted intercept: 2.9 μJ (not separately isolated).
ASCON-128 per-byte cost	Permutation rounds on the CPU. Measured: 1.180 μs and 0.089 μJ per byte.

Table 18-Resource cost before encryption

In short, AES-128-GCM trades a larger one-time hardware “boot-up” cost for a cheaper per-byte hardware execution cost, while ASCON-128 avoids that boot-up cost entirely but pays for every byte using slower, more energy-hungry general purpose CPU cycles. The break-even point is simply the payload size at which the energy saved by AES-128-GCM's cheaper per-byte hardware processing has caught up with, and overtaken, the energy it spent getting that hardware ready in the first place.

6.2.2 Objective 2 - Energy Cost Per Transmitted Byte Across the Full Duty Cycle

Objective 2 asked for the energy cost per byte across the duty cycle. Only the encryption phase was measured; the sleep phase was modelled as $3.3 \text{ V} \times I_{\text{sleep}} \times 60 \text{ s}$ (990 μJ for 5 μA , 1,980 μJ for 10 μA and wake-up and transmission were not measured. With this model, encryption was 0.4% of the cycle at 16 bytes and 13% at 1,600 bytes (ASCON-128, 5 μA case), and the choice of cipher changed total cycle energy by between 0.1% and 3.6% (Table 13). At small payloads the sleep term dominates both ciphers almost equally, so the choice matters little to total energy. It matters most for nodes that wake often, send large batches, or run close to their energy budget. Wi-Fi association and transmission, which were not measured, would enlarge the non-cipher part of the cycle further. The break-even is therefore a design rule for the active phase and not a large lever on battery life at low reporting rates.

This has a direct practical implication for objective 2 that would not have been visible from active-phase energy alone: at small, LPWAN-typical payload sizes, the sleep-phase energy term dominated the total duty cycle cost for both algorithms almost equally, meaning the choice of algorithm mattered comparatively little to total energy at very small payload. It was only once payload size grew large enough for the active-phase energy to become a meaningful share of the total that the algorithm choice identified in 4.5.2 began to matter in absolute terms.

6.2.3 Power versus time

The chapter 4 concluded that the energy difference was driven primarily by power rather than speed, because execution time differed significantly at only 21 of 100 sizes. That conclusion is withdrawn. The tests were one-tailed with the hypothesis "ASCON slower than AES", which holds only above the time crossover near 1,250 bytes, about 21 of the 100 sizes; two-sided, execution time differed at every size (5.5.6). In Campaign B, time dominates: ASCON-128 drew about 10% more extra power than AES-128-GCM (75.3 versus 67.8 mW), and at 1,600 bytes took 26% more time (1,927 versus 1,524 μs), which together give the 28% energy difference. At small payloads, AES-128-GCM with per-message setup is slower (102.8 versus 57.9 μs at 16 bytes), which is why ASCON-128 wins there. On this platform, energy differences between the ciphers are mostly differences in how long the CPU is busy.

6.2.4 Why the first analysis gave 347 bytes

The Campaign A break-even of 347 bytes was reproducible (345.0, 347.4 and 346.8 bytes without trimming and with two trimming factors) but was not confirmed by Campaign B. The comparison of 5.5 explains why: the measured AES power fell by 10.1 mW between 336 and 352 bytes while AES time rose smoothly, and the break-even lies exactly on that step; the power figures were 20 to 25 mW where 64 to 75 mW was measured; the timed region contained about 1.3 to 1.6 ms of instrumentation against a call of 58 to 103 μ s at 16 bytes; and the ciphers were collected in separate sessions. The disagreement between the regression (206 bytes) and the measured (347 bytes) break-even was a symptom of the same problem: a straight line fitted to data with steps. A further difference of about 325 μ s between the two ciphers' time offsets in Campaign A is much larger than any difference in Campaign B; since each Campaign A trial ran after a fresh boot, a plausible contributor is a first-execution effect. This was not tested and is proposed in 7.5.

6.3 Comparison with Existing Literature

Chapter 02 identified a conflict in the literature. Some studies reported ASCON as more efficient than AES, while others reported hardware accelerated AES as more efficient, with no study directly comparing the two on the same platform. Closer reading showed this was not really a conflict, the two groups of studies had answered different questions. Studies favoring ASCON, such as [Ridho Rabbani -, 2025] and [Tripathi et al., 2024], compared it against “Software-emulated” AES. Studies favoring hardware AES, such as [Rafat et al., 2025] and [Hung & Hsu, 2018], compared hardware AES against that same software baseline but never against ASCON.

The break-even measured in this study fills that gap: both groups of findings are correct within the scope of what they measured. ASCON-128 is the more efficient choice at small payloads when the AES context must be prepared per message, which is compatible with the lightweight-cryptography claim for short messages, and hardware-assisted AES-128-GCM is more efficient at larger payloads and, if its setup is amortised, at nearly all sizes, which is compatible with the accelerator studies. Software AES was not measured here, so the figures of those studies (for example the 74% latency reduction of ASCON over software AES) cannot be confirmed or refuted; only the qualitative relation is addressed.

Beyond this finding, the results agreed with the literature in two further respects. ASCON-128's execution time scaled almost perfectly linearly with payload size ($R^2 > 0.9999$, 5.4.2), matching its documented sponge-based structure, in which every block costs the same fixed number of CPU cycles [Ridho Rabbani -, 2025]. And AES-128-GCM's growing advantage at larger payloads matched the documented behavior of dedicated hardware engines maintaining stable, low-latency performance under load [Rafat et al., 2025].

The study also extended two specific gaps named in prior work. [Jin & Becker, 2022], whose benchmarking structure this study's experimental design was built on, measured only execution time and throughput, with no power measurement, energy analysis, or significance testing. This dissertation added all three. Separately, [Amirkhanova et al., 2026] explicitly named the absence of energy/CPU measurement and payload-size variation as limitations of their own AES-GCM/ESP32 pipeline; the per-byte energy analysis in Chapter 5 addresses both gaps.

Finally, the memory footprint result (AES-128-GCM using approximately 24% more RAM than ASCON-128, §4.6) is a new comparison rather than a confirmation of an existing one; no study reviewed in Chapter 2 compared the memory footprint of hardware-accelerated AES against a software AEAD cipher on the same platform. The direction of the gap is consistent with what each library must keep in memory: mbedTLS's AES-GCM context holds both the expanded AES key schedule and a precomputed GHASH table, while ASCON-128 only needs to keep its sponge state resident.

6.4 Limitations

The findings above should be read with the following constraints in mind. None of them are expected to change the direction of the break-even result, for the reasons given below, but they do bound how far the results can be generalized. All measurements were taken on a single board of ESP32-WROOM-32E unit. Due to the variation of the silicon chip of ESP32 the results are limited to generalize across other units. But this not affect the ASCON-vs-AES comparison itself, since both algorithms were measured on the same board under the same conditions.

The INA219 power sensor's 12-bit ADC sets a floor on the smallest power difference it can resolve. This affects confidence in the very smallest measured values, but the differences that drove the break-even result were well above this floor.

The firmware only instrumented the active encryption phase; the 990 μJ sleep-phase term was calculated analytically from a datasheet current figure and a fixed 60-second interval (4.2.4), rather than measured on the device. Because this term is identical for both algorithms at every payload size, it could not have shifted the break-even point but it does mean the total duty-cycle energy figures should be read as "measured active-phase energy plus a datasheet-derived sleep estimate," not as a fully empirical end-to-end measurement. For the same reason, this study cannot speak to duty cycles using a sleep interval other than 60 seconds.

The duty cycle's Wi-Fi/MQTT transmission step introduces variable-latency network activity outside the encryption operation itself. This is a plausible contributor to residual variance in the power measurements, though it should not have affected the execution-time measurement, which isolated only the encryption call. But it's a part of the IoT full duty cycle.

The brief power spike as Wi-Fi and the oscillator re-initialize after deep sleep reported elsewhere as roughly 22 μJ over 2.1 ms [Nkemeni et al., 2023] was not separately measured here. As with the sleep-phase estimate, this cost would apply equally to both algorithms and is not expected to bias the comparison, but it is a further respect in which the total energy figures are a supplemented estimate rather than a fully measured total.

"Energy per byte over one duty cycle" was used as the practical proxy for battery-life impact. It does not capture battery chemistry and discharge-curve effects, cumulative thermal effects, or RF-environment drift over a real deployment's lifetime. These were reasonable exclusions for a controlled laboratory comparison, but a field deployment (proposed in Chapter 6) would be needed to confirm the break-even point holds under those additional factors.

6.5 Summary of Chapter 6

The validated results show a payload-size break-even of about 100 bytes for AES-128-GCM with per-message setup, and none if the context is reused. The mechanism is a lower fixed cost but a higher per-byte cost for ASCON-128. The AES setup cost was

measured directly (30.9 μs , 1.90 μJ). The claim that power drives the difference is withdrawn. The 347-byte result of Campaign A is reproducible under trimming but is attributed to a step in the measured power, and was not confirmed. Against a modelled 60-second sleep interval, the cipher choice changed total energy by under 4%. The limitations of 6.4 bound these conclusions, and Chapter 7 states what should be done next.

Chapter 07: Conclusions and Future Work

7.1 Introduction

This chapter closes the dissertation: it summarizes whether the three objectives were met (7.2), answers the central research question in one paragraph (7.3), states this dissertation's contribution to the literature (7.4), and sets out recommended future work (7.5).

7.2 Achievement of Research Objectives

All three objectives set out in 1.8 were met.

- **Objective 01: energy per operation and per byte, and the initialization cost of AES-128-GCM** met. Energy per operation and per byte were measured for both ciphers (5.4.5), and the initialization step of the AES-GCM context was isolated: 30.9 μs and 1.90 μJ per call (5.4.3). Qualification: the fixed cost of ASCON-128 is a fitted intercept, not a separately measured step.
- **Objective 02: energy per byte across the duty cycle** - met, with a qualification. The active phase was measured and the sleep phase modelled (60 s, datasheet current). Encryption was 0.4% to 13% of the modelled cycle, and the cipher choice changed total energy by under 4% (5.4.8). Transmission and wake-up were not measured, so the study does not establish the energy of a fully measured duty cycle.
- **Objective 03: conditions favoring ASCON-128, including the break-even** - met. With AES-128-GCM setting up its context on every message the break-

even was about 100 bytes (95% interval 96 to 107 bytes); with the context reused AES-128-GCM was at least as efficient at every size (5.4.6 and 5.4.7).

7.3 Answer to the Research Question

Neither algorithm was unconditionally more energy-efficient on the ESP32; the deciding factors were payload size and how the AES context is managed. AES-128-GCM has a small per-message cost of preparing its context (30.9 μ s, 1.90 μ J) and a lower cost per additional byte (0.062 versus 0.089 μ J) than software ASCON-128. For a node that rebuilds the AES context on every message, software ASCON-128 was more energy-efficient below about 100 bytes and hardware-assisted AES-128-GCM above it, and at 1,600 bytes AES-128-GCM used 28% less energy per call. For a node that reuses the context, AES-128-GCM was more efficient at every size from 64 bytes and equal within measurement error at 16 bytes. AES-128-GCM used about 24% more run-time memory. Against a modelled 60-second sleep interval the difference in total energy per cycle was under 4%, so the choice matters most for nodes that send frequently or in large batches.

7.4 Contribution to Knowledge

Prior literature had compared ASCON with software AES, and hardware-accelerated AES with software AES, but not ASCON with hardware-assisted AES-128-GCM directly (2.7). This dissertation provides that comparison on one platform, and adds four things: a break-even of about 100 bytes together with the finding that it disappears when the AES context is reused; a direct measurement of the AES-GCM setup cost on the ESP32 (1.90 μ J); a documented comparison showing that a per-trial method with two sensor readings around a millisecond-scale call can give a reproducible but unsupported result, with a sustained-load method that resolves it; and a new comparison of run-time memory (AES-128-GCM about 24% larger).

7.5 Recommendations for Future Work

The limitations discussed in 5.4 point to the following concrete next steps:

- Multi-unit replication - repeat the measurement pipeline across multiple ESP32 boards to check whether the break-even point is a platform-level constant or varies between units.

- Empirically measure the sleep, wake-up and transmission phases with the INA219 (or a sensor with a suitable range) instead of a datasheet model, including the transmission of the tag and nonce overhead.
- Test other sleep intervals and messages per wake to check the practical-significance result of 5.4.8 directly.
- Use higher resolution current sensor – to check whether the INA219’s 12-bit ADC resolution floor artifacts the precision of the break-even estimate.
- Run a real battery-drain field trial - to validate the per-byte energy proxy used in this dissertation against actual measured device lifetime.

7.6 Closing Remarks

This dissertation compared software ASCON-128 with hardware-assisted AES-128-GCM on the ESP32 in a modelled IoT duty cycle. An early per-trial measurement gave a break-even of 347 bytes; a validation with a sustained-load method showed that the early method could not resolve the energy of the operation, and the conclusions were rebuilt on the second measurement. ASCON-128 is more energy-efficient for messages below roughly 100 bytes when the AES context is set up per message, and AES-128-GCM is at least as efficient otherwise, with a small effect on total energy at long sleep intervals. The result is offered as an evidence-based design rule for the active phase of energy-constrained IoT nodes, together with a documented method for measuring it.

8. References

- [Agugliaro et al., 2024] Agugliaro, F. M., Soto-Cruz, J., Ruiz-Ibarra, E., Vázquez-Castillo, J., Espinoza-Ruiz, A., Castillo-Atoche, A., & Mass-Sanchez, J. (2024). A Survey of Efficient Lightweight Cryptography for Power-Constrained Microcontrollers. *Technologies*.
<https://doi.org/10.3390/technologies>
- [Albert, 2026] Albert, B. (2026). *Energy-Efficient Cryptographic Protocols for Securing Low-Power IoT Networks*.
- [Arghire, 2024] Arghire, I. (2024, April 25). SecurityWeek. Retrieved from data protection:
<https://www.securityweek.com/ftc-sending-5-6-million-in-refunds-to-ring-customers-over-security-failures/>
- [Al-Mashhadani & Shujaa, 2022] Al-Mashhadani, M., & Shujaa, M. (2022). IoT Security Using AES Encryption Technology based ESP32 Platform. *International Arab Journal of Information Technology*, 19(2), 214–223.
<https://doi.org/10.34028/iajit/19/2/8>
- [Al-Shmailawi et al., 2026] Al-Shmailawi, H. A., Al-Hemiary, E. H., & Sikora, A. (2026). Evaluation of NIST Lightweight Cryptography Algorithms for Industrial Internet of Things. 49–55.
<https://doi.org/10.1109/idaacs68557.2025.11322258>
- [Amirkhanova et al., 2026] Amirkhanova, G., Ismailov, S., Amirkhanov, A., Adilzhanova, S., Zhasuzakova, M., & Chen, S.

- (2026a). A Lightweight, End-to-End Encrypted Data Pipeline for IIoT: An AES-GCM Implementation for ESP32, MQTT, and Raspberry Pi. *Information*, 17(1), 33. <https://doi.org/10.3390/info17010033>
- [Aydin et al., 2011] Aydin, N., Karaca, M., & Ercetin, O. (2011). *Energy-Optimal Scheduling in Low Duty Cycle Sensor Networks*. <http://arxiv.org/abs/1103.3616>
- [Beaulieu et al., 2013] Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., & Wingers, L. (2013). *The Simon and Speck Families of Lightweight Block Ciphers*
- [Bideh et al., 2020] Bideh, P. N., Sönnerup, J., & Hell, M. (2020, October 6). Energy consumption for securing lightweight IoT protocols. *ACM International Conference Proceeding Series*. <https://doi.org/10.1145/3410992.3411008>
- [Chen & Scholl, 2022] Chen, L., & Scholl, M. (2022, May 26). The Cornerstone of Cybersecurity – Cryptographic Standards and a 50 Year Evolution. Retrieved from NCCoE (National Cybersecurity Center of Excellence), NIST: <https://www.nccoe.nist.gov/news-insights/cornerstone-cybersecurity-cryptographic-standards-and-50-year-evolution>
- [Ciuffoletti, 2022] Ciuffoletti, A. (2022). Deep-Sleep for Stateful IoT Edge Devices. *Information (Switzerland)*, 13(3). <https://doi.org/10.3390/info13030156>
- [Community, 2026] Community, H. N. (2026). How to Profile ESP32 Power Consumption Accurately. Retrieved from Hubble Network Community: <https://hubble.com/community/guides/how-to-profile-esp32-power-consumption-accurately/>
- [Devadas et al., 2022] Devadas, S., Langowski, S., Samardzic, N., Servan-Schreiber, S., & Sanchez, D. (2022). Designing Hardware for Cryptography and Cryptography for Hardware. *Proceedings of the ACM Conference on*

- Computer and Communications Security*, 1–4.
<https://doi.org/10.1145/3548606.3559393>
- [Dezfouli et al., 2018] Dezfouli, B., Amirtharaj, I., & Li, C.-C. (2018). EMPIOT: An Energy Measurement Platform for Wireless IoT Devices. *Journal of Network and Computer Applications*, 121.
- [Elsadek, 2025] Elsadek, I. e. (2025). State-of-the-Art ASCON ASIC Achieving 4.3 Gbps@0.8V and 3.5Tb/J@0.6V Outperforming AES by 25 Times. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 72(8), 4021–4030. doi:10.1109/TCSI.2024.3522193
- [Faisal Abdullah Althobaiti, 2025] Faisal Abdullah Althobaiti. (2025). Lightweight cryptography for IoT: A comprehensive survey of algorithms, implementations, and standardization. *World Journal of Advanced Engineering Technology and Sciences*, 15(3), 516–525.
<https://doi.org/10.30574/wjaets.2025.15.3.0967>
- [Farooq et al., 2020] Farooq, U., Mushtaq, M., Bhatti, M., & Khurram Bhatti, M. (2020). *Efficient AES Implementation for Better Resource Usage and Performance of IoTs*.
<https://hal.archives-ouvertes.fr/hal-02979423>
- [Fernández-García & Jiménez-Fernández, 2024] Fernández-García, C., & Jiménez-Fernández, C. J. (n.d.). *Study and applications of the new NIST lightweight encryption with authentication standard in hardware implementations*.
- [Fortinet, 2024] Fortinet. (2024). *What is the CIA Triad and Why is it important?* Fortinet Cyberglossary. Retrieved from:
<https://www.fortinet.com/resources/cyberglossary/cia-triad>
- [Gerndt et al., 2025] Gerndt, M., Ispir, M., Nunez, I., & Benedict, S. (2025). Energy-Aware Duty Cycle Management for Solar-Powered IoT Devices. *Sensors*, 25(14).
<https://doi.org/10.3390/s25144500>

- [Hatch, 2025] Hatch, D. B. (2025). *Optimized Wireless Power Transmission for Low-Cost, Energy-Efficient Internet of Things Devices in Residential Environments*. <https://scholarsarchive.byu.edu/etd/10877>
- [Hung & Hsu, 2018] Hung, C. W., & Hsu, W. T. (2018). Power consumption and calculation requirement analysis of AES for WSN IoT. *Sensors (Switzerland)*, 18(6). <https://doi.org/10.3390/s18061675>
- [Ibrahim et al, 2025] Ibrahim, F., Rehman, A., Hassan, A., & Alzghoul, A. (2025). *Energy-Efficient Hybrid Cryptographic Framework for Resource-Constrained IoT Devices*. ISSN (e) 3007-3138 (p) 3007-312X. <https://doi.org/10.5281/zenodo.18047723>
- [Jin & Becker, 2022] Jin, Q., & Becker, M. (2022). *Performance Evaluation of Cryptographic Algorithms on ESP32 with Cryptographic Hardware Acceleration Feature*
- [Junchao Ma, 2009] Junchao Ma, W. Lou. (2009). *Energy Efficient TDMA Sleep Scheduling in Wireless Sensor Networks*. IEEE
- [Kandi et al., 2023] Kandi, A., Baksi, A., Gerlich, T., Guilley, S., Gan, P., Breier, J., Chattopadhyay, A., Ranjan Shrivastwa, R., Martinasek, Z., & Bhasin, S. (2023). *Hardware Implementation of ASCON*. <https://github.com/aneeshkandi14/ascon-hw-public>
- [Khriji et al., 2022] Khriji, S., Chéour, R., & Kanoun, O. (2022). Dynamic Voltage and Frequency Scaling and Duty-Cycling for Ultra Low-Power Wireless Sensor Nodes. *Electronics (Switzerland)*, 11(24). <https://doi.org/10.3390/electronics11244071>
- [Kietzmann et al., 2021] Kietzmann, P., Boeckmann, L., Lanzieri, L., Schmidt, T. C., & Ahlisch, M. W. (n.d.). *A Performance Study of Crypto-Hardware in the Low-end IoT*. Retrieved from: <https://github.com/>

- [Kim et al., 2020] Kim, K., Choi, S., Kwon, H., Kim, H., Liu, Z., & Seo, H. (2020). PAGE-Practical AES-GCM encryption for low-end microcontrollers. *Applied Sciences (Switzerland)*, 10(9).
<https://doi.org/10.3390/app10093131>
- [Kozłowski & Sosnowski, 2019] Kozłowski, A., & Sosnowski, J. (2019). Energy Efficiency Trade-Off Between Duty-Cycling and Wake-Up Radio Techniques in IoT Networks. *Wireless Personal Communications*, 107(4), 1951–1971.
<https://doi.org/10.1007/s11277-019-06368-0>
- [Kumar & Sethi Ericsson, 2019] Kumar, S., & Sethi Ericsson, M. (2019). *Internet of Things Security: State of the Art and Challenges*.
<https://trustee.ietf.org/license-info>
- [Mac et al., n.d.] Mac, W.-F., Baseband, W.-F., Le Link Controller, B., & Le Baseband, B. (n.d.). *ESP32-C3 Series Datasheet*
- [Mahdi & Abdullah, 2025] Mahdi, L. H., & Abdullah, A. A. (2025). A Hybrid Post-Quantum Cryptographic Framework Integrating Kyber-512 and ASCON for Secure IoT Communications. *Engineering, Technology and Applied Science Research*, 15(5), 26527–26533.
<https://doi.org/10.48084/etasr.12471>
- [Matas Pipiras, 2024] Matas Pipiras. (2024). *Performance Analysis of AES Modes of Operation on an ESP32-based Platform - A Comparison of Execution time and RAM Usage in IoT Context*
- [Mitchel et al., 2025] Mitchel, R., Ryan, J., Kaiser, A. M., & Hoiness, G. W. (2025). Throughput of ASCON Compared with Popular IoT Encryption Algorithms. *Military Cyber Affairs*, 8.
<https://digitalcommons.usf.edu/mca/vol8/iss1/3>
- [Nkemeni et al., 2023] Nkemeni, V., Mieveville, F., & Tsafack, P. (2023). Energy Consumption Reduction in Wireless Sensor Network-Based Water Pipeline Monitoring Systems

- via Energy Conservation Techniques. *Future Internet*, 15(12). <https://doi.org/10.3390/fi15120402>
- [Padmanabha1 et al., 2025] Padmanabha, P., Srinivas, S., Sunil, S., Yasha, D., & Shirur, J. M. (2025). ASIC-Oriented RTL Implementation of ASCON-128 for IoT Devices. *International Journal of Science and Engineering Invention (IJSEI)*, 11(10). <https://doi.org/10.23958/ijsei/vol11-i10/298>
- [Radhakrishnan et al., 2024] Radhakrishnan, I., Jadon, S., & Honnavalli, P. B. (2024). Efficiency and Security Evaluation of Lightweight Cryptographic Algorithms for Resource-Constrained IoT Devices. *Sensors*, 24(12). <https://doi.org/10.3390/s24124008>
- [Rafat et al., 2025] Rafat, S. H., Jarin, M. N., Mahdee, T. M., Miah, A. S. M., Haque, N., Tusher, M. M. R., Shin, J., & Ahmed, I. (2025). Lightweight Cryptographic Algorithm Analysis for Secure IoT Communication on ESP-32 Platforms. *2025 IEEE International Conference on Quantum Photonics, Artificial Intelligence, and Networking, QPAIN 2025*. <https://doi.org/10.1109/QPAIN66474.2025.11171742>
- [Ridho Rabbani, 2025] Ridho Rabbani, M. (2025). *Performance Benchmarking Analysis of ASCON and AES-128 Lightweight Cryptography Primitives on ESP32 Hardware*.
- [Sadhu et al., 2022] Sadhu, P. K., Yanambaka, V. P., & Abdelgawad, A. (2022). Internet of Things: Security and Solutions Survey. *Sensors*, 22(19). MDPI. <https://doi.org/10.3390/s22197433>
- [Schneider, 2024] Schneider, J. (2024). *The History of Cryptography*. IBM Think. Retrieved from: <https://www.ibm.com/think/topics/cryptography-history>

- [Sinha, 2025] Sinha, S. (2025, October). *State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally*. IoT Analytics
- [Syahmi Md Dzahir & Seng Chia, 2023] Syahmi Md Dzahir, M. A., & Seng Chia, K. (2023). of ESP32 Microcontroller for Real-Time MQTT IoT-Based Monitoring System. *2023 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies, 3ICT 2023*, 255–261. <https://doi.org/10.1109/3ICT60104.2023.10391358>
- [Texas Instruments, 2008] Texas Instruments. (2008). *INA219 Data Sheet*. www.ti.com
- [F. Thabit, 2023] F. Thabit, O. C.-G. (2023). Cryptography Algorithms for Enhancing IoT Security. 100759.
- [Trap, 2025] Trap, C. (2025, January 24). *Securing the Digital Age: A Deep Dive into Data Encryption Techniques*. Canary Trap. Retrieved from: <https://canarytrap.com/blog/securing-the-digital-age/>
- [Tripathi et al., 2024] Tripathi, Dr. D. R., Akhtar, H. N. N., & Bhandarkar, R. R. (2024). Enhancing IOT Security: Investigating Lightweight Encryption Algorithms for Resource-Constrained Devices. *International Journal for Research in Applied Science and Engineering Technology*, 12(10), 350–357. <https://doi.org/10.22214/ijraset.2024.64519>
- [Turan et al., 2025] Turan, M. S., McKay, K. A., Chang, D., Kang, J., & Kelsey, J. (2025). *Ascon-based lightweight cryptography standards for constrained devices*. NIST. <https://doi.org/10.6028/NIST.SP.800-232>

